# 嗯 Qt 样式使用说明

## 一、声明：

### 1. 文档说明：

- 学习 Qt 样式表前提：

  如果你了解 CSS，那么我相信这对你来说会很容易理解并应用于实际界面美化，它与 CSS 有些相似之处，当然如果同时也了解 Qt，那么你会很快学会如何运用 Qt 样式为程序设计出漂亮的界面。

- 阅读本说明文档：

  首先一、阅读声明，二、名词解释，三、语法简介，六、附属例题解释，每次实验并对照着五、样式表参照表，体会并学习如何运用样式表里关键词。

- 样式表参照表之间的关系很复杂：

  很多需要重复描述的选择器、属性、值，但由于本文档不能跳转，所以要自己去查找，按照它语法关系层层迭代定位表格，并按照字母表顺序定位要查找内容的大概范围。
  这里的表格以 Qt help 文档为准。

- 暂不介绍：

  为了尽快完成本文档的初步可使用的目的，很多在本程序中暂时用不到的窗口部件暂未解释介绍，窗口部件介绍表格中会提到“暂不介绍”，但是辅助控制器，属性，值都是完全展示在表格中。

- 附加的例题格式说明：

  例子是经过实验得出的结论，用 C 语言的注释方式添加到相应的样式代码后面，当然你的文件( .CSS) 同样满足这个注释方式。

  /* 注释内容 */

  /*imagesForExample: example_for_XXX000.png */实例图片

- 如何做实验：

  Qt 源码 bin 目录下的 designer.exe,运行/拖入控件/右键/改变样式表/应用；
  Qt Creator 也可以。

- 遇到问题请尽快联系作者

### 2. 背景介绍：

Qt 为图形界面应用程序提供一个完整的 C++应用程序开发框架。

Qt 的样式表主要是受到 CSS 的启发，通过调用 QWidget::setStyleSheet() 或 QApplication::setStyleSheet()，你可以为一个独立的子部件、整个窗口，甚至是整个应用程序指定一个样式表。样式表由影响窗口部件绘制的样式规则组成。这些规则都是普通文本。由于在运行时会解析样式表，所以可以通过定制样式表的方式来尝试设计不同的 Qt 应用程序。

www.docin.com

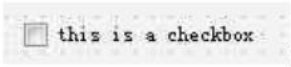
## 二、名词解释：

### 1. 选择器（selector）

意思是：选择特定的类，一般为一个可以定制样式表的 Qt 类，所有可以作为选择器的 Qt 类都在五、样式参照表：**2.** 可以应用样式表的窗口部件表=选择器中列出，选择器的格式参照五、样式参照表： **1** 样式选择器类型表。

所谓的选择器可以理解为 CSS 中的选择器，他指定了一类部件进行设计。

### 2. 辅助控制器（sub-control）

辅助控制器 一词是相对于选择器存在的，可以理解为我们选择了一个部件，例如一个 QCheckBox， 

这个部件它分为两个部分，文本部分和可以点击的小窗口的部分。而这个可点击的小窗口部分我们要单独的设置，就要再次分离出来，就需要::indicator(QCheckBox 有这个辅助控制器)来设置，如下例题：

```
QCheckBox::indicator{
    width:20px;
    height:20px;
}
/*说明是在 QCheckBox 中的指示符（indicator）宽为 20px，height 为 20px。*/
```

辅助控制器是用 :: 双冒号进行指定。

如果没有::indicator 那么我们这个小例子将是对整个 QCheckBox 设置的，使用了辅助控制器的时候就自动分离出这个小窗口，对小窗口进行设置。

不同的选择器有不同的辅助控制器，具体可查看五、样式表参照表：**3.** 辅助控制器列表中详细介绍相应的辅助控制器在不同的类中应用，详细说明在类中的什么位置。

### 3. 状态（pseudo-states）

除了辅助控制器对一个部件的分离，样式表还可以根据窗口部件的各个状态来设置窗口。例如 hover 表示鼠标划过时的状态，例子如下：

```
QCheckBox:hover{
    color: red;
}
/*例子说明只有当鼠标滑过复选框文本时变为 red*/
```

状态是用 ： 冒号进行区分每一个状态。

更详细的状态列表在五、样式表参照表：**3** 状态列表中查找

状态可以多个一起使用，也可以和辅助控制器一起使用，这样设置窗口部件的时候分的会更加详细。

如下小例子:

```
QCheckBox:hover,QCheckBox:checked{
```

```
    color:red;
}
QPushButton::hover{
    color:red;
}
```

## 4. 属性

它是一个窗口部件所固有的特征、性质，每一个窗口部件都会有属于他们自己的属性。如前面做的小例子中我们一直未曾提过 color,width,height 等。组合多个属性同时使用设计出多种效果。五、样式表参照表：**4** 属性列表查找有更多的详细介绍。

## 5. 值

是属性 ：后面跟随的一组数字，颜色或者是一个 bool 类型等这些我们称它为值，这些值决定了窗口部件的最终的展示效果。
查看值的表达方式
五、样式表参照表：**4** 值列表

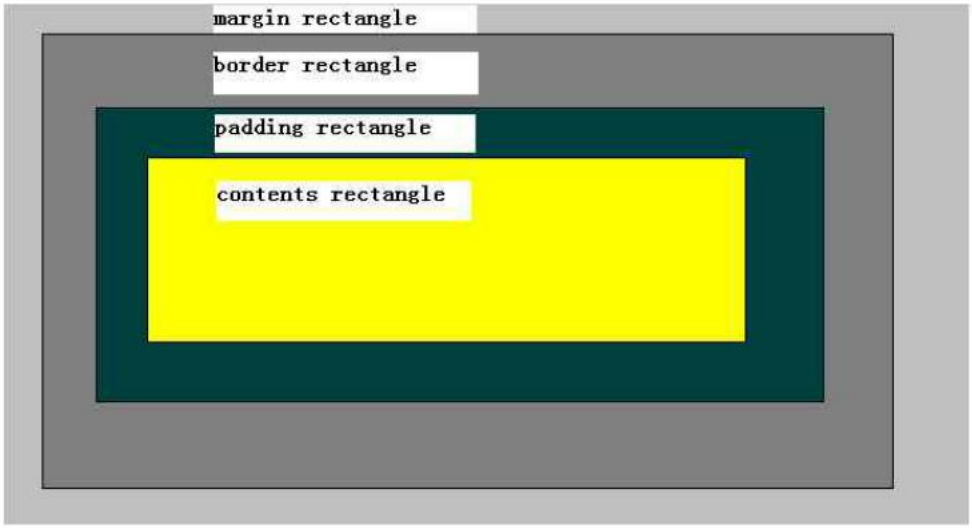
## 6.逻辑否（!）

有时候我们在设置某种状态的属性时，希望同时在某些非（!）的状态下设置，这个时候我们就要用（!）来选择某种状态，比如!checked 、!has-children（没有子目录）等等。

## 7. 盒模型（The Box Model）

这个模式指定了 4 个影响布局的矩形，从而绘制一个自定义的窗口部件。
1.**Content** rectangle 是最里面的矩形，它绘制窗口部件内容（如文字，图片）的地方。
2.**padding** rectangle 包围 content rectangle。它负责由 **padding** 属性指定填充操作。主要是窗口部件内容与边缘线(border)之间的空隙，它可以用 top，right，bottom 和 left 设置它的大小。
**3.border** rectangle 包围 padding rectangle。它为边界预留空间。可以认为是窗口的外框线。下面讲的分割图形的方法中把 border 当做是一个区域来理解的。参考四、高级应用：九宫格分割法
4.**margin** rectangle 最外面的矩形，他包围 border rectangle，负责指定的边缘空白区域，主要是负责与其他的窗口部件的距离。
如果没有指定他们四个，则默认是四个重合在一起的。
如图：

www.docin.com




## 8.角弧度

窗口部件四个角弧度。radius 设置角的弧度，如 border-radius:4px;
角的弧度是 4px。

## 9.背景色和前景色

部件的前景色用于绘制窗口部件上面的文本，可以通过 color 属性指定。

背景色用于绘制窗口部件的填充矩形，可以通过 background-color 属性指定。

背景图片使用 background-image 属性定义，它用于绘制由 background-origin 指定在盒模式中四个区域的图片开始显示的起点位置。背景图片在盒模式域内的对齐和平铺方式可以通过 background-position 和 background-repeat 属性指定。

如果指定的背景图片具有 alpha 通道（透明效果），通过 background-color 指定的颜色将会透过透明区域。在 background-color 属性中有介绍。

## 10.（#）

指定某一个按钮，#号后面是指定类的对象名。要知道代码才能运用。

# 三、语法介绍与问题解决：

## 1.语法

- 选择器 {
  属性 : 值 ;
  }
  QPushButton{

```
    color: red;
}
```

- 选择器 : 状态{
    属性 : 值 ;
 }

```
QPushButton : hover{
    color : red;
 }
```

- 选择器 :: 辅助控制器{
    属性 : 值;
 }

```
QCheckBox:: indicator{
    color: red;
 }
```

- 选择器 :: 辅助控制器 , 选择器 :: 辅助控制器{
    属性 : 值;
 }

```
 QTabBar::tab:selected, QTabBar::tab:hover {
 background: qlineargradient(x1: 0, y1: 0, x2: 0, y2: 1,
                                 stop: 0 #fafafa, stop: 0.4 #f4f4f4,
                                 stop: 0.5 #e7e7e7, stop: 1.0 #fafafa);
}
```

- 选择器 : 状态 , 选择器 : 状态{
    属性 : 值;

```
}
QPushButton:hover, QPushButton:pressed{
    color : red;
}
```

- 选择器 , 选择器 , 选择器{
    属性 : 值;
 }

```
 QLineEdit, QPushButton, QCheckBox{
     color : red;
 }
```

你可以任意的组合，当然这只是简单的组合，还有更加复杂的组合。

1.可以先查看一下选择器的表达方式，考虑一下是只对一个类进行操作还是，对整个界面操作，仔细阅读整个表格 1.样式选择器类型表确定应用哪种样式表。

2.选择器的格式确定后，就要确定你要所要针对的具体的类型，那么就参照 2.可以应

www.docin.com

用样式表的窗口部件。

3. 之后就查看你所要应对的选择器里的部分进行设置，就要再去查看辅助控制器。

4. 再分的细一点，搞清楚一个窗口部件分为几个状态，鼠标划入，点击，关闭...他们决定了你所要设置状态的属性设置。

5. 定位了前面的，就要改变他的特性了，也就是进行属性设置，查找属性表，对应属性表超找对应的值表，也有可能还要通过值表的迭代（值的值还需要一个方式表达。）最终知道这个值是数字，还是一个颜色，或者是一个 bool 类型，抑或是其他的关键词(如 padding、content...)。仔细阅读他们之间的关系。

## 2.遇到问题

也许你已经组合了很多的样式表，但是有时候你会发现，有时候有些属性值不起作用，或者说图像变形，并不是你所要看到的效果，不要着急。

1. 查看一下你的语法是否正确，如果你保证确保无误的话，那么就想一想，是否是在构建这个窗口控件之前进行设置的，这个会影响到你的属性是否被读入。参考六、附属例题解释：32. 定制 QTool 查看是否被其他的属性覆盖。当一个属性被具有同一选择器的几个规则设置时，那么只有最后一个规则起作用（这是一个难点）。

2. 查看是否有相应的关联的属性已被设为 bool=1。

3. 图片无法显示：查看路径是否正确在 Qt 中是”/”代替 window 下的”\”，使用相对路径，本应用程序的相对路径（是相应程序读取这个样式 css 文件），当然你也可以写绝对路径的方式读取图片，但是路径这个方法是不提倡的。

4. 大小变化的窗口控件是否背景图片选择了 border-image。详细参考四、高级应用与六、附属例题解释：34.QPushButton 与 image

# 四、高级应用：

www.docin.com

## 1.九宫格分割法：

之所以叫做九宫格分割法是源于，把边界图分为 3X3 的小格，当填充窗口部件时如图 A，4 个角保持不变，如图 B,

| | | |
|---|---|---|
| A | B | C |
| D | E | F |
| G | H | T |

A：原始图

| | | |
|---|---|---|
| A | B | C |
| D | E | F |
| C | H | T |

B：结果图

其他的 5 个格子被拉伸或者平铺填充可用的空间。

使用 border-image 属性可以指定各个边界图，他要求指定一个图像文件名和定义 9 个格子的 4 条“切线”。切线用到上、右、下和左边缘的距离定义。Border.png 作为边界图，距离上、右、下和左的切线为 4、8、12 和 16 应该如下定义：

border-image：url（boder.png） 4 8 12 16;

但使用边界图时，必须显示地设置 border-image 属性。一般情况下，border-width 应该与切线的位置一致；否则，为了与 border-width 相符合，角上的格子将被拉伸或者缩短。对 border.png 应该这样设置指定他的四个边框的宽度：

border-width: 4px 8px 12px 16px;

这样才能把刚才切割的四个角的图片完全的放置到 border 中当做 border 区域的背景。

## 2.渐变器

Qt 支持三种渐变器查看例题 5.渐变器

1.线性渐变（qlineargradient）：连接这两点的线上有一系列的颜色断点。两个控制点之间的不同位置指定不同的颜色。位置用 0 和 1 之间的浮点数来确定，0 对应着第一个控制点，1 对应着第二个控制点。两个指定断点之间的颜色由线性插值得出。

2.辐射渐变（qradialgradient）：有一个中心点（xn，yn），半径 r，一个焦点（xf，yf）以及颜色断点定义。

中心点和半径定义一个圆。颜色从焦点向外扩散，焦点可以是中心点或者圆内的其它点。

3.梯形渐变（qconicalgradient）：由一个中心点（xn，yn）和一个角度 a 定义。颜色在中心点周围像钟表的秒针扫过一样扩散。

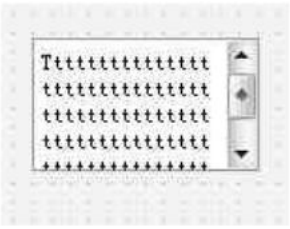
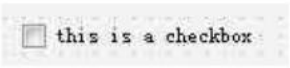
# 五、样式表参照表：

## 1.样式选择器类型表

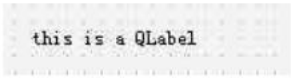
| 选择器 | 实例 | 可以匹配的窗口部件 |
|---|---|---|
| 全局对象 | * | 对全局的任意窗口部件 |
| 类型 | QDial | 给定类的实例，包括这个类的子类 |
| 类 | .QDial | 给定类的实例，不包括子类 |
| 标识 | QDial#ageDial | 给定对象名称的窗口(ageDial 表示对象的名字，在代码中可知) |
| Qt 属性 | QDial[y=”0”] | 为某些属性赋值的窗口，表示当 Y（属性）为 0（false）或者为 1 （true）的时候 |
| 子对象 | QFrame > QDial | 给定窗口部件的直接子类 |
| 子孙对象 | QFrame QDial | 给定窗口部件的子孙窗口部件 |

## 2.可以应用样式表的窗口部件表 = 选择器

| 部件 | 可应用的样式 |
|---|---|

| | |
|---|---|
| QAbstractScrollArea<br>  <br>上图为一个 QTextEdit 文字不能完全显示而自动添加的滚动条 | 1.支持盒模型。<br>2. 所有 QAbstractScrollArea （QAbstractItemView、QGraphicsView、QMdiArea、QPlainTextEdit、QScrollArea、QTextEdit）的派生类，包括 QTextEdit 和 QAbstractItemView(带 item 的类)，用 background-attachment 属性支持背景可滚动的，设置 background-attachment 属性为 fixed 则提供一个 background-image 背景图片不跟随滚动条滚动，反则设置 background-attachment 属性为 scroll 则是背景图片可以跟随滚动条滚动。<br>3.具体看例：6 定制 QAbstractScrollArea。 |
| QCheckBox<br>  | 1.支持盒模型，<br>2.可以用 ::indicator 辅助控制器设置 check 指示符(一个可以选择的小方块)，默认时，check 指示符放置在部件矩形内容的左上角处。<br>3.spacing 属性指定 check 指示符和文本(右侧的描述文字)的距离。<br>4.具体看例：7 定制 QCheckBox。 |
| QColumnView | 1.可以被 image 属性控制样式，arrow(箭头)指示符可以被 ::left-arrow 和 ::right-arrow 辅助控制器样式控制。 |
| QComboBox<br>  | 1.支持盒模型。<br>2.下拉按钮(drop-down button)样式可以被 ::drop-down 辅助控制器设置，默认情况下，下拉按钮填充矩形部件的右上侧，下拉按钮的箭头标记(arrow mark)通过样式 ::down-arrow 辅助控制器控制。默认的情况下，箭头是位于下拉按钮的中间位置。<br>3.具体看例：8 定制 QComboBox。 |
| QDateEdit<br>  | 具体看例：24 定制 QSpinBox。 |
| QDateTimeEdit<br>  | 具体看例：24 定制 QSpinBox。 |
| QDialog<br>  | 1.只支持 background、background-clip 和 background-origin 属性。<br>Warning：确保我们定义了宏 Q_OBJECT 在我们定制的部件里。 |
| QDialogButtonBox<br>上图 QDialog 对话框中的按钮 | 可以用 button-layout 属性改变按钮的布局。 |
| QDockWidget | 1.当浮动窗口停靠的时候，支持标题栏和标题栏按钮。<br>2.border 属性可以控制浮动窗口的边框。 |

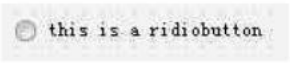
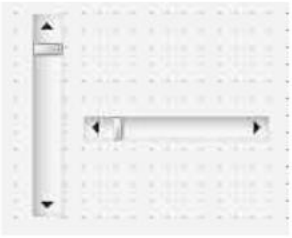
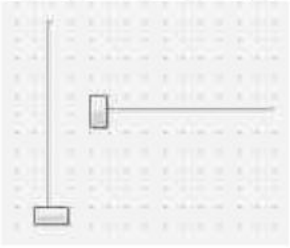
www.docin.com

| | |
|---|---|
|  | 3.::title 辅助控制器可以定制标题栏样式。<br>4.::title 辅助控制器上的 ::close-button 控制浮动窗口关闭按钮属性，float-button 浮动控制按钮。<br>5.当标题栏可视的时候，:vertical 状态被设置，当然这个状态取决于浮动窗口的 3 种状态设置。即 :floatable、:closable 和 :movable 状态。<br>6.Note：用 QMinWindow::separator 去设计大小处理。<br>7.Warning：设置的样式不能应用于未停靠的窗口上。<br>8.具体看例：9 定制 QDockWidget。 |
| QDoubleSpinBox | 具体看例：24 定制 QSpinBox。 |
| QFrame | 1.支持盒模式。<br>2.自 Qt 版本 4.3 开始，在 QLabel 上设置一个样式表将自动设置 QFrame::frameStyle 属性到 QFrame::StyledPanel。<br>3.具体看例：10 定制 QFrame。 |
| QGroupBox<br>  | 1.支持盒模式。<br>2.::title 辅助控制器设置标题样式（图片中“GroupBox”字样）。默认的情况下，标题位置取决于 QGroupBox::textAlignment.<br>3.在可点击选择的情况下，标题包含 check 指示符用 ::indicator 辅助控制器设置。<br>4.Spacing 属性是控制 check 指示符与文本描述的空间。<br>5.具体看例：11 定制 QGroupBox。 |
| QHeaderView<br>  | 1.支持盒模式。<br>2.::section 辅助控制器设置标题视图，section 辅助控制器支持 :middle、:first、:last、:only-one、next-selected、:previous-selected、:selected 和 :checked 状态。<br>3.::up-arrow 和 ::down-arrow 设置排序的样式。<br>4.具体看例：15 定制 HeaderView。 |
| QLabel<br>  | 1.支持盒模式。<br>2.不支持鼠标悬停状态(鼠标经过的时候)。<br>3.自 Qt 版本 4.3 开始，在 QLabel 上设置一个样式表将自动设置 QFrame::frameStyle 属性到 QFrame::StyledPanel。<br>4.具体看例：10 定制 QFrame（QLabel 类继承自 QFrame）。 |
| QLineEdit<br>  | 1.支持盒模式。<br>2.selection-color 和 selection-background-color 分别设置选择时的背景颜色和和前景色。<br>3.lineedit-password-character 的属性设置密码类型。<br>4.具体看例： 13 定制 QLineEdit。 |
| QListView | 1.支持盒模式。<br>2.当表格交替颜色是可以选择的，表格交替颜色设置用 alternate-background-color 属性设置。<br>3.选择的颜色 selection-color 和 selection-background-color 控制选择颜色和选择的背景颜色。 |

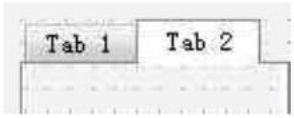
www.docin.com

| | |
|---|---|
| | 4.show-decoration-selected 属性控制选择行为。<br>5.::item 更加细致的控制 QListView 的 items。<br>6.具体看例： 6 定制 QAbstractScrollArea。<br>具体看例： 14 定制 QListView。 |
| QListWidget<br>  <br> | 具体看例： 14 定制 QListView。 |
| QMainWindow<br>（主窗口） | 1.支持样式分割器。<br>2.在 QMainWindow 用分割器时，QDockWidget 用::separator 辅助控制器来设置样式。<br>2.具体看例： 15 定制 QMainWindow。 |
| QMenu<br>（菜单） | 1.支持盒模式。<br>2.::item 辅助控制器设置每一个 items，除此之外还支持状态如 :selected、:default、 :exclusive 和 non-exclusive 状态。<br>3.::indicator 辅助控制器设置菜单的可点击的属性。<br>4.菜单分隔风格样式由::separator 辅助控制器设置。<br>5.right-arrow 和 left-arrow 设置子目录箭头是向右还是向左。<br>6.::scroller 设置滚动条。<br>7.::tearoff 设置 tear-off（拆卸下来成为独立的窗口）<br>8.具体看例： 16 定制 QMenu。 |
| QMenuBar<br>（菜单栏） | 1.支持盒模式。<br>2.spacing 属性设置两个 item 的距离。<br>3.::item 辅助控制器控制每一个 items 的样式。<br>4.具体看例： 17 定制 QMenuBar。 |
| QMssageBox<br>（弹出来的报告错处，提示的对话框） | Messagebox-text-interaction-flags 属性用来修改消息框中的文本 |
| QProgressBar<br>  <br> | 1.支持盒模式。<br>2.::chunk 辅助控制器设置块状结构样式(进度条里面的颜色设置可能是一段一段的)的进度。条块是显示在盒模时的 content 中。<br>3.进度条中如果显示文字，text-align 属性设置进度条中文字位置。<br>4.:indeterminate 不确定状态设置。<br>具体看例： 18 定制 QProgressBar。 |
| QPushButton<br>  <br> | 1.支持盒模式。<br>2.:default、:flat(外框是否突起)、:checked 状态。<br>3.当按钮在菜单栏里时，用::menu-indicator 辅助控制器设置菜单栏的风格。<br>4.用:open 和 :closed 定制是否可点击的按钮。<br>5.例如：QPushButton{ |

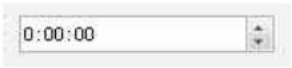
www.doc88.com

| | |
|---|---|
| | background-color：red；border：none；<br>}<br>6.具体看例： 19 定制 QPushButton。 |
| QRadioButton<br>  | 1.支持盒模式。<br>2.可以用 ::indicator 辅助控制器设置 check 指示符(一个可以选择的小圆圈)，默认时，check 指示符放置在矩形内容的左边。<br>3.spacing 属性指定 check 指示符和文本(右侧的描述文字)的距离。<br>4.具体看例： 20 定制 QRadioButton。 |
| QScrollBar<br>  | 1.支持盒模式。<br>2.控件上盒模式横纵的 content 的内容依赖于槽上的滑动器（hangle）的移动。<br>3.QScrollBar 的范围（长或者宽由 scroll 的方向确定）由 width 和 height 设置，用:horizontal 和:vertical 设置方向。<br>4.::handle(句柄)辅助控制器宽或者高（设置 min-width 或者 min-height）的最小宽度和最小高度，取决于它的方向。<br>5.::add-line 设置添加内容的按钮样式。依赖于箭头的方向。::right-arrow 和::down-arrow,箭头默认是在 add-line 的盒模式 contents 的中间位置。<br>6.::sub-line 设置减少内容的按钮样式。::left-arrow 和::up-arrow，箭头在按钮的中间位置。<br>7.::sub-page 减少页和 add-page 增加页。<br>.具体看例： 21 定制 QScrollBar。 |
| QSizeGrip<br>(通过它可以改变它所在顶级窗口的大小，在顶级窗口的右下角) | 1.支持 width、height、image 属性。<br>2.具体看例： 22 定制 QSizeGrip。 |
| QSlider<br>  | 1.支持盒模式。<br>2.对于横向，min-width 和 height 属性必须提供。<br>3.对于纵向，min-height 和 width 属性必须提供。。<br>4.::groove 设置槽，一般情况下槽是默认在盒模式的 content 中，控件可以滑动的 handle(句柄)用::handle 辅助控制器设置。句柄在槽上移动，在盒模式中的 content。<br>5.具体看例： 23 定制 QSlider。 |
| QSpinBox<br>  | 1.支持盒模式。<br>2.::up-button 和 up-arrow 控制上面的按钮和按钮上箭头。<br>3.::down-button 和 down-arrow 控制下面的按钮和按钮上的箭头。默认箭头在按钮的中间位置。<br>具体看例： 24 定制 QSpinBox。 |
| QSplitter<br>(界面分离器)<br>一个界面被分为两个 | 1.支持盒模式。<br>2.::handle 设置分割器可移动句柄 handle。<br>具体看例： 25 定制 QSplitter。 |
| QStatusBar<br>（状态栏） | 1.只有 background 属性起作用，每一个 items 用::items 设置。<br>2.具体看例： 26 定制 QStatusBar。 |

www.docin.com

| | |
|---|---|
| QTabBar<br>  <br> | 1.每一个 tab 页用::tab 设置样式，关闭用 ::close-button,tab 页支持 only-one、:fist、:last、:middle、:privious-selected、:next-seleced、:selected 状态。<br>2.:top、:left、:right、:bottom 状态取决于 tabs 在部件的方向。<br>3.选择重叠 tabs 状态（这个状态时表示在很多 tab 的时候，点击一个 tab 页为当前显示时，就表示这个 tab 突出覆盖了左右 tab 的 n 个 px）是利用负 margins（盒模式中的 margin）或者是相对位置来控制。<br>4.当 tab 很多个的时候，有时候 tabbar 不能把它完全显示出来，就会在 tabbar 上显示可以左右调节 tab 页的按钮，这个按钮为 QToolButton，定制这个 QToolButton 的宽度用::scroller 辅助控制器定制 。<br>5.::tear 定制 tear 指示器的样式。<br>6.QTabBar 的 tab 方向用 alignment（top\|bottom\|left\|right\|center）设置。<br>6.改变 QTabWidget 的 QTabBar 方向用 tab-bar 辅助控制器。<br>7.具体看例： 27 定制 QTabWidget 和 QTabBar。 |
| QTabWidget<br>  <br> | 1.用::pane 设置页面的外框。<br>2.::left-corner 和 ::right-corner 设置左侧角落和右侧角落。<br>3.它的 tab 位置通常用::tab-bar 设置，默认是 QWindowsStyle 的风格。<br>4.:top, :left, :right, :bottom 选择 tab 上、左、下和右的方位。<br>5.具体看例： 27 定制 QTabWidget 和 QTabBar。 |
| QTableView | 1.支持盒模式。<br>2.alternating row colors 可用的时候我们可以使用属性 alternate-background-color 控制交替的表格背景颜色。<br>3.selection-color 和 selection-background-color 属性设置选择 item 的前景颜色和背景颜色。<br>4.在 QTableView 中控件可以作为一个 QAbstractButton 和可以用 QTableView QTableCornerButton::section 设置样式。<br>5.表格颜色 gridline-color。<br>6.参看 QAbsractScrollArea 设置背景 backgrounds。<br>具体看例： 28 定制 QTableView。 |
| QTableWidget<br>  | 具体看例： 28 定制 QTableView。 |
| QTextEdit<br>（一个文本编辑器） | 1.支持盒模式。<br>2.selection-color 和 selection-background-color 决定它的文本颜色和背景颜色。 |

www.doc.com

| | |
|---|---|
| | 具体看例： 6 定制 QAbstractScrollArea。 |
| QTimeEdit<br>  <br> | 具体看例： 24 定制 QSpinBox。 |
| QToolBar<br>（工具栏） | 1.支持盒模式。<br>2.:top, :left, :right, :bottom 工具栏的位置，是在主窗口的上，左，右，下状态设置属性。<br>3.:first, :last, :middle, :only-one 选择了工具栏中第一组按钮。<br>4.工具栏的分隔符是由::separator 控制。<br>5.可以控制工具栏可移动::handle 选择器选择它并改变样式。<br>6.具体看例： 29 定制 QToolBar。 |
| QToolButton<br>（工具栏按钮） | 1.支持盒模式。<br>2.如果 QToolBox 有一个菜单，::menu-indicator 来选择它的指示符（就是可以移动工具栏的句柄，可以参考 WPS 的新建工具栏）。<br>3. 当 QToolBar 在 QToolButton::MenuButtonPopup 模式下，::menu-button 控制这个菜单按钮，::menu-arrow 是控制这个按钮的箭头，默认的情况下会放置在按钮上 content 位置。<br>4.当 QToolBar 显示箭头时候::up-arrow, ::down-arrow, ::left-arrow and ::right-arrow 可以被用到。<br>5.具体看例： 30 定制 QToolButton。 |
| QToolBox<br>  <br><br>如同好友 QQ 模式 | 1.支持盒模式。<br>2.::tab 辅助控制器设置 tab,tab 支持:only-one、:first、:、last:、middle:、previous-selected:next-selected、:selected 状态。 |
| QToolTip<br>（鼠标划过时的提示语） | 1.支持盒模式。<br>2.支持 opacity（不透明）属性。<br>具体看例： 10 定制 QFrame（QToolTip 类继承自 QFrame）<br>31 定制 QToolTip。 |
| QTreeView | 1.支持盒模式。<br>2.当 alternating row colors（行交替着不同的颜色） 是可用的时候，可以通过 alternate-background-color 属性设置它交替颜色。<br>3.selection-color 属性和 selection-background-color 属性设置选择 item 的前景颜色和背景颜色。<br>4.show-decoration-selected 设置选择行为属性。<br>5.分支辅助控制器::branch 支持:open, :closed, :has-sibling and :has-children 状态。 |

www.docin.com

| | |
|---|---|
| | 6.::item 辅助控制器更详细的控制 QTreeView 的 items。<br>7.看 QAbsractScrollArea 关于可滚动背景的描述。<br>具体看例： 32 定制 QTreeView。 |
| QTreeWidget<br>  | 具体看例： 32 定制 QTreeView。 |
| QWidget | 1.只支持 background, background-clip 和 background-origin 属性。<br>2.如果你的类是QWidget的子类,你需要重写paintEvent函数如下：<br>void CustomWidget::paintEvent(QPaintEvent *)<br>{<br>QStyleOption opt;<br>opt.init(this);<br>QPainter p(this);<br>style()->drawPrimitive(QStyle::PE_Widget, &opt, &p, this);<br>}<br>/*如果没有样式表设置，那么它是什么作用都不起的*/<br>Warning：确保我们定义了宏 Q_OBJECT 在我们定制的部件里。 |

## 3. 辅助控制器列表

| 辅助控制器 | 说明 |
|---|---|
| **::add-line** | 在 QScrollBar 中添加一行的按钮。 |
| **::add-page** | 在 QScrollBar 中添加页的按钮。 |
| **::branch** | QTreeView 的分支。 |
| **::chunk** | QProgressBar 的进度条里的块（里面的进度条可以变成一块一块的增加，而不是整体都是一种颜色）。 |
| **::colse-button** | QDockWidget 和 QTabBar 标题栏上的关闭按钮。 |
| **::corner** | 在 QAbstractScrollArea 两个滚动条之间的位置。 |
| **::drwn-arrow** | QComboBox、QHeaderView(排序时需要)、QScrollBar、QSpinBox 的向下箭头。 |
| **::down-button** | QScrollBar、QSpinBox 的向下按钮。 |
| **::drop-down** | QComboBox 展开时。 |
| **::float-button** | QDockWidget 标题栏上的浮动按钮。 |
| **::groove** | QSlider 的槽。 |
| **::indicator** | QAbstractitemView、QCheckBox、QRadioButton、可点击的 QMenu 的 item、可点击的 QGroupBox 的指示符。 |

www.docin.com

| | |
|---|---|
| **::handle** | QScrollBar、QSplitter、QSlider 的滑块。 |
| **::icon** | QAbstractitemView 和 QMenu 的图标。 |
| **::item** | QAbstractitemView、QMenuBar、QMenu、QStatusBar 的单独的一项。 |
| **::left-arrow** | QScrollBar 的向左的箭头。 |
| **::left-corner** | QTabWidget 的左侧。 |
| **::menu-arrow** | 菜单里 QToolButton 箭头。 |
| **::menu-button** | 工具栏上的按钮。 |
| **::menu-indicator** | 菜单里的 QPushButton 指示符。 |
| **::right--arrow** | QMenu 或者是 QScrollBar 的右侧箭头。 |
| **::pane** | QTabWidget 去掉标题的框架。 |
| **::right-corner** | QTabWidget 的右侧。 |
| **::scroller** | QMenu 和 QTabBar 因为界面大小布局左右调试的滚动按钮。 |
| **::section** | QHeaderView 的表头横向和纵向。 |
| **::separator** | QMainWindow 和 QMenu 的分离器（就是一个主窗口被分割成几个小的区域的线，QMenu 里是 item 的分离线）。 |
| **::sub-line** | QScrollBar 内容减少方向的按钮。 |
| **::sub-page** | QScrollBar 减少一页的按钮，在滑块与减少一行 sub-line 之间。 |
| **::tab** | QTabBar 和 QToolBox 的一个页选项。 |
| **::tab-bar** | 一个 QTabWidget 的 tab 按钮，设置 tabs 一般用::tab。 |
| **::tear** | TabBar 的指示符 |
| **::tearoff** | QMenu 的指示符 |
| **::text** | QAbstractitemView 的内容。 |
| **::title** | QGroupBox 和 QDockWidget 的标题。 |
| **::up-arrow** | QHeaderView（排序时）、QScrollBar、QSpinBox 向上按钮箭头。 |
| **::up-button** | QSpinBox 的向上按钮。 |

www.docin.com

## 4. 状态列表

| 状态 | 说明 |
|---|---|
| :active | 当前活动的窗口。 |
| :adjoins-item | QTreeView 的一个 branch 存在毗邻下一个与自己不是兄弟项目的项。 |
| :alternate | 当 QAstractitemView 的 QAstractitemView::alternatingRowColors()的属性设置为 true 时，行之间背景颜色交替颜色变化， |
| :bottom | 在 item 的下面，例如 QTabBar 的 tab 按钮在下面。 |
| :checked | 按钮已选中。 |
| :closable | items 是可以关闭的，例如 QDockWidget 有一个 QDockWidget::DockWidgetClosable 的功能。 |
| :closed | （open 相对的）窗口位于关闭或者销毁的状态，例如 QTreeView 没有打开时的状态 |

| | |
|---|---|
| :default | 默认的状态。 |
| :disabled | 禁用窗口部件 |
| :editabled | QComboBox 可以编辑的 |
| :edit-focus | 那种可编辑的控件，比如文本框，当它正在编辑的时候，就是 QStyle::State_HasEditFocus 状态。 |
| :enabled | 启用窗口部件。 |
| :exclusive | 表示按钮组设置为单选，只能选择一个的状态。例如菜单栏的选项。 |
| :first | 部件的第一个，例如 QTabBar 的第一个 tab |
| :flat | 没有突起的部件。 |
| :floatable | 部件是可浮动，例如 QDockWidget。 |
| :focus | 窗口部件有输入焦点。 |
| :has-children | Item 有子目录的，例如 QTreeView。 |
| :has-siblings | 有兄弟目录的，例如 QTreeView。 |
| :horizontal | 部件是横向的。 |
| :hover | 鼠标划过窗口部件时的状态。 |
| :indeterminate | 按钮部分被选中的状态。 |
| :last | 最后一个，例如：QTabBar 的最后一个 tab。 |
| :left | 位于左面，例如 QTabBar 的 tab 位于左面的那个。 |
| :maximized | 最大化状态。 |
| :middle | 在列表中中间位置，例如 QTabBar 的 tab 不是最后一个也不是第一个的。 |
| :minimized | 最小化的时候。 |
| :movable | 这个部件可以移动，例如 QDockWidget。 |
| :no-frame | 这个部件是没有 frame 的，例如 QLineEdit 和 QSpinBox。 |
| :non-exclusive | 不能单选的，菜单来就是单选的。 |
| :off | 前提是这个部件是可以选或者不选的，与 on 相对，窗口部件是“off”状态。 |
| :on | 前提是这个部件是可以选或者不选的，与 off 相对，窗口部件是“on”状态。 |
| :only-on | 只有一个，例如 QTabBar 只有一个 tab 的时候。 |
| :open | 窗口部件位于打开或者是展开的状态，如 QTreeView，QComboBox 和 QPushButton 在 menu 时的打开状态。 |
| :next-selected | 选择下一个的时候，在列表中，例如 QTabBar。 |
| :pressed | 鼠标按键点击窗口部件 |
| :previous-selected | 选择上一个，例如 QTabBar。 |
| :read-only | 部件是只读的，例如 QLineEdit,和 QComboBox 的编辑窗口为不可编辑时。 |
| :right | 部件位置的右侧，QTabBar 的 tab 可以位于右侧。 |
| :selected | 被选择的，例如 QTabWidget 被选择的 tab 页或者是 QMenu 下拉菜单被选择的某一个条目。 |
| :top | 位于部件的上面，例如 QTabBar 的 tab 可以位于上面，也可以在下面一样。 |
| :unchecked | 按钮未被选中的。 |

www.docin.com

| | |
|---|---|
| :vertical | 垂直的方向。 |
| :window | 这个部件是一个窗口。 |

## 5.属性列表

| 属性 | 值 | 说明 |
|---|---|---|
| alternate-background-color | Brush | 表示交替论表格颜色的属性，它应用于QAbatractitemView的子类，<br>如果你不设置这个颜色值，它会有一个默认的值备用。<br>QTreeView {<br>alternate-background-color: blue;<br>background: yellow;<br>}<br>你也可以参考background<br>和selection-background-color |
| background | Background | 简单的设置背景颜色，等价于backgroun-color,background-image,background-repeat或者backgroun-position.<br>这个属性支持QAbstractitemView的子类，子类包括（QAbstractSpinBox子类,QCheckBox,QDialog,QFrame,QGroupBox,QLabel,QLineEdit,QMenu,QMenuBar,QPushButton,QRadioButton,QSplitter,QTextEdit,QToolTip,和plain的QWidgets）<br>通常，它需要设置Qt::BrushStyle样式一个填充模式，你可以用background-color属性为三个渐变器来和Qt::SolidPattern设置样式。<br>QLabel { background-color: yellow }<br>QLineEdit { background-color: rgb(255, 0, 0) }<br>可以参考：<br>background-origin,selection-background-color,background-clip,background-attachment,alternete-background-color. |
| background-color | Brush | 用于部件的背景颜色。<br>例如：<br>QLabel{ background-color: yellow }<br>QLineEdit{background-color: rgb(255, 0, 0) } |
| background-image | Url | 用于部件的背景图片，可以用background-color的般透明度给图片加效果。<br>QFrame { |

doc(n)2.com
www.doc(n)2.com

| | | |
|---|---|---|
| | | background-image: url(:/images/hydro.png) } |
| background-repeat | Repeat | 无论是否指定重复的填充背景图片(background-image)于盒模式的矩形内，只要图片太小它都会自动的重复填充。如果这个 y 或者 x 属性没有被指定，背景图片是从两边(x.y)同时开始重复。例：<br>QFrame {<br>background: white url(e:/wb/styles/start.png);<br>background-repeat: repeat-y;<br>background-position: right;<br>}<br>这样我们指定了背景图片它重复的方向是 y（竖着）轴，右侧开始。 |
| backgroud-position | Alignment | 在盒模式中定位背景图片开始显示的位置，默认是左上角开始显示。<br>例如：<br>QFrame {<br>background: url(:/images/footer.png);<br>background-position: bottom left;<br>}<br>这样就可以设置部件显示图片是从左下角开始显示。 |
| background-attachment | Attachment | 描述背景图片是否在 QAbstractScrollArea 的区域跟随滚动条滚动，scrolled 是图片跟随滚动条滚动，fixed 是图片不跟随滚动条滚动。<br>QTextEdit {<br>background-image: url("leaves.png");<br>background-attachment: fixed;<br>}<br>可以参考 background |
| background-clip | Origin | 1.在部件的盒模式中那些可以当做背景显示的。<br>2.这个属相指定盒模式的 backgroud-color 和 background-image 被裁减。<br>3.这个属性支持以下类<br>QAbstractItemView 的子类, QAbstractSpinBox subclasses, QCheckBox, QComboBox, QDialog, QFrame, QGroupBox, QLabel, QPushButton, QRadioButton, QSplitter, QTextEdit, QToolTip, 和一些无复杂设置的 QWidgets.<br>4.如果没有使用该属性，那么默认情况下是 |

| | | |
|---|---|---|
| | | border。<br>5.例如：<br>QFrame {<br>background-image:<br>url(:/images/header.png);<br>background-position: top left;<br>background-origin: content;<br>background-clip: padding;<br>}<br>6.可参考 background, background-origin 和 The Box Model. |
| background-origin | Origin | 1.可以 background-position 和 background-image 联合一起使用设置空间的背景显示在盒模式中的起点位置。<br>2.默认是支持类 QAbstractItemView 的子类, QAbstractSpinBox subclasses, QCheckBox, QComboBox, QDialog, QFrame, QGroupBox, QLabel, QPushButton, QRadioButton, QSplitter, QTextEdit, QToolTip, 和一些设置比较单一的 QWidgets.<br>3.默认情况下是 padding。<br>4.例子：<br>QFrame {<br>background-image:<br>url(:/images/header.png);<br>background-position: top left;<br>background-origin: content;<br>}<br>5.可以参考 background 和 The Box Model. |
| border | Border | 1.简便的设置部件的外框线。<br>2.等价于 border-color, border-style, 或者 border-width.<br>5.这个属性支持 QAbstractItemView 的子类, QAbstractSpinBox subclasses, QCheckBox, QComboBox, QFrame, QGroupBox, QLabel, QLineEdit, QMenu, QMenuBar, QPushButton, QRadioButton, QSplitter, QTextEdit, QToolTip, 和一些设置比较单一的 QWidgets.<br>3.例子：<br>QLineEdit { border: 1px solid white } |
| border-top | Border | 简单的设置部件上的外框线，等效于 border-top-color, border-top-style, 或者 border-top-width |
| border-right | Border | 简单的设置部件上的右外框线，等效于 |

doc 豆丁
www.docin.com

| | | |
|---|---|---|
| | | border-right-color, border-right-style,或者 border-right-width |
| border-bottom | Border | 简单的设置部件上的左外框线，等效于 border-bottom-color, border-bottom-style, 或者 border-bottom-width |
| border-left | Border | 简单的设置部件的下外框线，等效于 border-left-color, border-left-style,或者 border-left-width |
| border-color | Box Colors | 1.设置所有外框线的颜色，border-top-color, border-right-color, border-bottom-color, 和 border-left-color.<br>2.支持所有的 QAbstractItemView 的子类, QAbstractSpinBox subclasses, QCheckBox, QComboBox, QFrame, QGroupBox, QLabel, QLineEdit, QMenu, QMenuBar, QPushButton, QRadioButton, QSplitter, QTextEdit, QToolTip, 和一些设置比较单一的 QWidgets.<br>3.默认情况下一般默认为和前景色一致。<br>4.例子：<br>QLineEdit {<br>border-width: 1px;<br>border-style: solid;<br>border-color: white;<br>}<br>5.可以参考 border-style, border-width, border-image, and The Box Model |
| border-top-color | Brush | 外框线的上面颜色。 |
| border-right-color | Brush | 外框线的右面颜色。 |
| border-bottom-color | Brush | 外框线的下面颜色。 |
| border-left-color | Brush | 外框线的左面颜色。 |
| border-image | Border Image | 1.把图片的分割的九个区域中的四个角区域填充到 border 中（详细查看九宫格分割法），其实 border 在盒模式中是一块区域，默认值只是因为太小了，所以我们通常认为是线，但是在这里我们要改正一下我们以往的惯用思维。<br>2.支持所有 QAbstractItemView 的子类, QAbstractSpinBox subclasses, QCheckBox, QComboBox, QFrame, QGroupBox, QLabel, QLineEdit, QMenu, QMenuBar, QPushButton, QRadioButton, QSplitter, QTextEdit and QToolTip.<br>3.可以参考 border-color, border-style, border-width, an 和 The Box Model. |
| border-radius | Radius | 1.外框线 border 角的弧度， |

| | | |
|---|---|---|
| | | 2.等效于<br>border-top-left-radius,<br>border-top-right-radius,<br>border-bottom-right-radius, 和<br>border-bottom-left-radius.<br>3.默认的情况下是 0px.<br>4.例子：<br>QLineEdit {<br>border-width: 1px;<br>border-style: solid;<br>border-radius: 4px;<br>}<br>5.可以查看 border-width 和 The Box Model. |
| border-top-left-radius | Radius | 左上角角的弧度。 |
| border-top-right-radius | Radius | 右上角角的弧度。 |
| border-bottom-right-radius | Radius | 左下角角的弧度。 |
| border-bottom-left-radius | Radius | 右下角角的弧度。 |
| border-style | Border Style | 1.对所有的 border 设置。<br>2.支持所有的类 QAbstractItemView 的子类, QAbstractSpinBox subclasses, QCheckBox, QComboBox, QFrame, QGroupBox, QLabel, QLineEdit, QMenu, QMenuBar, QPushButton, QRadioButton, QSplitter, QTextEdit, 和 QToolTip.<br>3.默认情况下是 none。<br>4.例子：<br>QLineEdit {<br>border-width: 1px;<br>border-style: solid;<br>border-color: blue;<br>}<br>5. 可以查看 border-color, border-style, border-image, 和 The Box Model. |
| border-top-style | Border Style | border 的上外框的样式。 |
| border-right-style | Border Style | border 的右外框的样式。 |
| border-bottom-style | Border Style | border 的下外框的样式。 |
| border-left-style | Border Style | border 的左外框的样式。 |

www.docin.com

| | | |
|---|---|---|
| border-width | Box Lengths | 1.border 的宽度值。<br>2.等效于 border-top-width, border-right-width, border-bottom-width,和 border-left-width.<br>3.支持所有的类 QAbstractItemView 的子类, QAbstractSpinBox subclasses, QCheckBox, QComboBox, QFrame, QGroupBox, QLabel, QLineEdit, QMenu, QMenuBar, QPushButton, QRadioButton, QSplitter, QTextEdit, 和 QToolTip.<br>4.例子<br>QLineEdit {<br>border-width: 2px;<br>border-style: solid;<br>border-color: darkblue;<br>}<br>5.可以参考 border-color, border-radius, border-style, border-image, 和 The Box Model. |
| border-top-width | Length | border 的上边缘线的宽度。 |
| border-right-width | Length | border 的右边缘线的宽度。 |
| border-bottom-width | Length | border 的下边缘线的宽度。 |
| border-left-width | Length | border 的左边缘线的宽度。 |
| bottom | Length | 1.相对位置下（默认的情况下）辅助控制器的移动就抵消了它原来的位置，<br>指定 bottom y 等效于 top：-y。(看 TabWidget)。<br>2.如果位置是绝对的，辅助控制器的 bottom 关联于母控件的边界。<br>3.例子：<br>QSpinBox::down-button { bottom: 2px }<br>4.可以参考 left, right, and top。 |
| button-layout | Number | 1.对类 QDialogButtonBox 和 QMessageBox 的按钮进行布局。<br>2.提供的值是 0（WinLayout），1（MacLayout），2（KdeLayout），3（GnomeLayout）(不同系统值不一样)<br>3.默认情况下是当前情况下 SH_DialogButtonLayout 的风格。<br>4.例子：<br>* { button-layout: 2 } |
| color | Brush | 1.字体的颜色，认为是前景色。<br>2.支持 QWidget:: palette 都可以支持的属性。<br>3.这个属性没有被设置默认为黑色。 |

www.doc88.com

| | | |
|---|---|---|
| | | 4.例子：<br>QPushButton { color: red }<br>5.可以参看 background 和 selection-color. |
| dialogbuttonbox-buttons-have-icons | Boolean | 1.是否显示按钮图标。<br>2.设置为1则显示图标，设置为0则图标不显示。<br>3.例子：<br>QDialogButtonBox{<br>dialogbuttonbox-buttons-have-icons: 1; }<br>Note：在使用这个属性之前一定要确保 QDialogButtonBox 被创建，所以最好应用在父界面上或者主程序里。 |
| font | Font | 1.简单的对界面的字体进行设置。<br>2.等效于 font-family, font-size, font-style, 及 font-weight.<br>3.那么就支持所有的部件（部件支持 font）使用。<br>4.默认的情况下是 QWidget::font.<br>5.例子：<br>QCheckBox {<br>font: bold italic large "Times New Roman"<br>} |
| font-family | String | 1.字体集<br>2.例子：<br>QCheckBox {<br>font-family: "New Century Schoolbook"<br>} |
| font-size | Font Size | 1.文本字体的大小。<br>2.在 Qt 中只支持 pt 和 px。<br>3.例子：<br>QTextEdit { font-size: 12px } |
| font-style | Font Style | 1.文本字体的样式。<br>2.例子：<br>QTextEdit { font-style: italic } |
| font-weight | Font Weight | 文本字体的磅数。 |
| gridline-color* | Color | 1.在 QTabelView 的表格颜色。<br>2.如果这个属相没被设定，默认值被当前样式 SH_Table_GridLineColor 指定。<br>3.例子：<br>* { gridline-color: gray } |
| height | Length | 1.辅助控制器的高度（例如一个 widget）。<br>2.如果没有设定，则默认值是根据当前辅助控制器自己的样式或者是窗口部件的样式显示。<br>3.Warning:如果一个部件设定了固定的宽和高， |

www.docin.com

| | | |
|---|---|---|
| | | 那么这个属性将不起做用，如设置了一个部件的 min-height 和 max-height。<br>4.例子：<br>QSpinBox::down-button { height: 10px }<br>5.可以参考 width。 |
| icon-size | Length | 1.在部件中设置图标的高和宽。<br>2.以下的类可以用到这个属性<br>QCheckBox<br>QListView<br>QPushButton<br>QRadioButton<br>QTabBar<br>QToolBar<br>QToolBox<br>QTreeView |
| image* | Url+ | 1.在辅助控制器的盒模式中的 content 中绘制图形。<br>2.这个属性接受一个 Urls 列表或者是一个 svg<br>3.设置了一个辅助控制器的图片，默认的情况下是设置了这个选择器的宽和高。<br>4.4.3 中和更高的版本中，可以指定图片开始显示位置。<br>5.对于辅助控制器的noly-将不支持其他的类型元素。<br>6.Warming：QIcon 的 SVG 插件需要的是 SVG 图片。<br>7.QSpinBox::down-button{<br>image: url(:/images/spindown.png)<br>} |
| image-position | alignment | 在 4.3 中和更高的版本中，可以指定图片开始显示位置。 |
| left | Length | 1.相对位置下（默认）辅助控制器的移动就抵消了它原来的位置向右。<br>2.绝对位置下，关联于母控件的 left。<br>3.默认的情况下是 0.<br>4.例子：<br>QSpinBox::down-button { left: 2px }<br>5.可以参考 right, top, 和 bottom. |
| lineedit-password-character* | Number | 1.QLineEdit 的密码，是 Unicode 格式。<br>2.默认是 SH_LineEdit_PasswordCharacter 属性的样式。<br>3.例子：<br>* { lineedit-password-character: 9679 } |
| margin | Box Lengths | 1.部件的间距。 |

doc 资料 www.docin.com

| | | |
|---|---|---|
| | | 2.等效于 margin-top, margin-right, margin-bottom, and margin-left.<br>3.支持的类有 QAbstractItemView 的子类, QAbstractSpinBox subclasses, QCheckBox, QComboBox, QFrame, QGroupBox, QLabel, QLineEdit, QMenu, QMenuBar, QPushButton, QRadioButton, QSplitter, QTextEdit, 和 QToolTip.<br>4.默认是 0。<br>5.例子:<br>QLineEdit { margin: 2px }<br>6.可以参考 padding, spacing, The Box Model. |
| margin-top | Length | 上面的空白区域。 |
| margin-right | Length | 右面的空白区域。 |
| margin-bottom | Length | 下面的空白区域。 |
| margin-left | Length | 左面的空白区域。 |
| max-height | Length | 1.辅选择控制器的最大高度。<br>2.支持所有的 QAbstractItemView 所有子类 QAbstractSpinBox subclasses, QCheckBox, QComboBox, QFrame, QGroupBox, QLabel, QLineEdit, QMenu, QMenuBar, QPushButton, QRadioButton, QSizeGrip, QSpinBox, QSplitter, QStatusBar, QTextEdit, 和 QToolTip.<br>3.默认只依赖于当前的部件盒模型的 content。<br>4.例子: QSpinBox { max-height: 24px }<br>5.可以参考 max-width. |
| max-width | Length | 1.辅选择控制器的最大宽度。<br>2.支持所有的 QAbstractItemView 所有子类 QAbstractSpinBox subclasses, QCheckBox, QComboBox, QFrame, QGroupBox, QLabel, QLineEdit, QMenu, QMenuBar, QPushButton, QRadioButton, QSizeGrip, QSpinBox, QSplitter, QStatusBar, QTextEdit, 和 QToolTip.<br>3.默认只依赖于当前的部件盒模型的 content。<br>4.例子: QSpinBox { max-width: 72px }<br>5.可以参考 max-height. |
| messagebox-text-interaction-flags* | Number | 1.一个文本交互的一个行为。它的值依赖于 Qt::TextInteractionFlags。<br>2.默认是 SH_MessageBox_TextInteractionFlags 样式。<br>3.例子:<br>QMessageBox { |

www.doc88.com

| | | |
|---|---|---|
| | | messagebox-text-interaction-flags: 5 } |
| min-height | Length | 1.辅选择控制器的最小高度。<br>2.支持所有的 QAbstractItemView 所有子类 QAbstractSpinBox subclasses, QCheckBox, QComboBox, QFrame, QGroupBox, QLabel, QLineEdit, QMenu, QMenuBar, QPushButton, QRadioButton, QSizeGrip, QSpinBox, QSplitter, QStatusBar, QTextEdit, 和 QToolTip.<br>3 默认只依赖于当前的部件和模型的 contents。<br>4 例子：QSpinBox { mix-height: 24px }<br>5.可以参考 mix-width. |
| min-width | Length | 1.辅选择控制器的最小宽度。<br>2.支持所有的 QAbstractItemView 所有子类 QAbstractSpinBox subclasses, QCheckBox, QComboBox, QFrame, QGroupBox, QLabel, QLineEdit, QMenu, QMenuBar, QPushButton, QRadioButton, QSizeGrip, QSpinBox, QSplitter, QStatusBar, QTextEdit, 和 QToolTip.<br>3.默认只依赖于当前的部件盒模型的 content。<br>6.例子：QSpinBox { max-widht: 72px }<br>可以参考 min-height. |
| opacity* | Number | 1.部件的不透明度，（0[transparent]~255[opaque]）.<br>2.只支持 tooltips<br>3.如果没有设置样式则依赖于 SH_ToolTipLabel_Opacity。<br>4.例子：<br>QToolTip { opacity: 223 } |
| padding | Box Lengths | 1.部件的 padding。<br>2.等效于 padding-top, padding-right, padding-bottom, 和 padding-left。<br>3.支持类 QAbstractItemView 的子类，QAbstractSpinBox subclasses, QCheckBox, QComboBox, QFrame, QGroupBox, QLabel, QLineEdit, QMenu, QMenuBar, QPushButton, QRadioButton, QSplitter, QTextEdit, 和 QToolTip.<br>4.例子：<br>QLineEdit { padding: 3px }<br>5.可以参考 margin, spacing, 和 The Box Model. |

doc 文档 www.docin.com

| | | |
|---|---|---|
| padding-top | Length | 部件的上面的 padding。 |
| padding-right | Length | 部件的右面的 padding。 |
| padding-bottom | Length | 部件的下面的 padding。 |
| padding-left | Length | 部件的左面的 padding。 |
| paint-alternating-row-colors-for-empty-area | bool | 控制是否可以绘制颜色交替的 QTreeView 的空白区域。 |
| position | relative \|absolute | 用相对的还是绝对的控制 left，right，top 和 bottm 的位置，默认的情况下是相对的。 |
| right | Length | 1.如果是相对位置（默认）辅助控制器的移动抵消了原来的位置向左。<br>2.绝对位置，子控件的右边界关联于母控件的右边界。<br>3.默认的情况下是 0.<br>4.例子：<br>QSpinBox::down-button { right: 2px }<br>5.可以参考 left, top, 和 bottom. |
| selection-background-color* | Brush | 1.选择一个 item 时的背景颜色。<br>2.这个部件是可选并且 QWidget::palette 都支持此属性。<br>3.默认情况下是 Highlight。<br>4.例子：<br>QTextEdit { selection-background-color: darkblue }<br>5.可以参考 selection-color 和 background. |
| selection-color* | Brush | 1.选择一个 item 时的前景色。<br>2.这个部件是可选并且 QWidget::palette 都支持此属性。<br>3.默认是 HighlightedText<br>4.例子：<br>QTextEdit { selection-color: white }<br>5.可以查看 selection-background-color and color. |
| show-decoration-selected* | Boolean | 1.<br>2.默认情下 SH_ItemView_ShowDecorationSelected 控制样式。<br>3.例子：<br>* { show-decoration-selected: 1 } |
| spacing* | Length | 1.部件的内部空间大小。<br>2.支持类 QCheckBox, checkable QGroupBoxes, QMenuBar, and QRadioButton。<br>3.默认值由当前的部件的大小决定。<br>4.例子：<br>QMenuBar { spacing: 10 } |

doc 豆丁 www.docin.com

| | | |
|---|---|---|
| | | 5. 可以参考 padding 和 margin. |
| **subcontrol-origin*** | Origin | 1.在盒模式的四个区域中设置辅助控制器起点位置。<br>2.默认为 padding。<br>3.例子：<br>QSpinBox::up-button {<br>image: url(:/images/spinup.png);<br>subcontrol-origin: content;<br>subcontrol-position: right top;<br>}<br>4.可以参考 subcontrol-position. |
| **text-align** | Alignment | 1.设置文本，图标在部件盒模式 content 中文本起始方向。<br>2.例子：<br>QPushButton {<br>text-align: left;<br>}<br>Note：这个属性目前（4.8）只支持 QPushButton 和 QProgressBar. |
| **text-decoration** | none<br>underline<br>overline<br>line-through | 添加文本效果。没有效果，下划线，上划线，线穿过文本。 |
| **top** | Length | 1.如果是相对位置（默认）辅助控制器的移动通过抵消原来的位置向下移动。<br>2.绝对位置，子控制器的上边界关联于母控件的上边界线。<br>3.默认的情况下是 0.<br>4.例子：<br>QSpinBox::up-button { top: 2px }<br>5.可以参考 right, left, 和 bottom. |
| **width** | Length | 1.辅助控制器的宽度（例如一个 widget）。<br>2.如果没有设定，则默认值是根据当前辅助控制器自己的样式显示。<br>3.Warning:如果一个部件设定了固定的宽和高，那么这个属性会不起做用的，如设置了一个部件的 min-height 和 max-height。<br>4.例子：<br>QSpinBox::up-button { width: 12px }<br>5.可以参考 height。 |

www.doin.com

## 6. 图标列表

暂不介绍。

## 5. 值列表

阅读此表方式：
这是属性的值，通过属性查找到这里时就按照值查找后面对应的语法，把语法里的黑体的首字母非大写的直接加在后面。语法一栏里仍然有黑体字体，仍然按照这个方式查找，直到没有黑色首字母大写值，参看例题。

| 值 | 语法 | 说明 |
| --- | --- | --- |
| Alignment | { top<br>\| bottom<br>\| left<br>\| right<br>\| center } | 方位 |
| Attachment | { scroll<br>\| fixed }* | 背景是否跟随滚动条滚动 |
| Background | { Brush<br>\| Url<br>\| Repeat<br>\| Alignment }* | 任意的一个代替在查找下一层（笔刷，路径，重复，方位）。 |
| Border | { Border Style<br>\| Length<br>\| Brush }* | 简单的设置线的属性，长度，笔刷， |
| Boolean | 0\|1 | True（1）false（0） |
| Border Image | none<br>\| Url Number{4}<br>(stretch \|<br>repeat){0,2} | 看九宫格分割法 |
| Border Style | dashed<br>\| dot-dash<br>\| dot-dot-dash<br>\| dotted<br>\| double<br>\| groove<br>\| inset<br>\| outset<br>\| ridge<br>\| solid<br>\| none | 用这些值绘制一个 border |
| Box Colors | Brush{1,4} | 可以设置四个方向的颜色顺序为上，右，下，左，缺 |

www.docin.com

| | | |
|---|---|---|
| | | 省方式参考下面 Box Lengths。 |
| Box Lengths | Length{1,4} | 四个方向顺序为 top, right, bottom, and left<br>注释皆为此顺序。<br>例：<br>1.QLabel {<br>border-width: 1px }<br>/* 1px 1px 1px 1px */<br>当输入一个值如 1px 则默认 QLable 的四个方向的值都为 1px。<br>此时是 right 默认用 top 的值，bottom 自动用 top 的值，right 自动用 right 的<br>2.QLabel {<br>border-width: 1px 2px }<br>/* 1px 2px 1px 2px */<br>输入两个值则表示 bottom 值和 right 的都使用默认的值 bottom 使用 top，right 使用 left。<br>3.QLabel {<br>border-width: 1px 2px 3px }<br>/* 1px 2px 3px 2px */<br>输入三个值表示最后一个 left 值默认用 right 侧的值<br>4.QLabel {<br>border-width: 1px 2px 3px 4px }<br>/* 1px 2px 3px 4px */ |
| Brush | Color<br>\| Gradient<br>\| PaletteRole | 指定颜色\|渐变\|调色板 |
| Color | rgb(r, g, b)<br>\| rgba(r, g, b, a)<br>\| hsv(h, s, v)<br>\| hsva(h, s, v, a)<br>\| #rrggbb<br>\| Color Name | 颜色 rgba 和 hsva 后面的参数是透明度。 |
| Font | (Font Style \|<br>Font Weight)<br>{0,2}<br>Font Size String | 简单设置文字字体 |
| Font Size | Length | 文字字体大小 |
| Font Style | normal<br>\| italic<br>\| oblique | 文字字体的格式 |
| FontWeight | normal<br>\| bold<br>\| 100 | 文字的磅<br>bold 加粗 |

docin 豆丁 www.docin.com

| | | |
|---|---|---|
| | \| 200<br>...<br>\| 900 | |
| Gradient | qlineargradient<br>\| qradialgradient<br>\| qconicalgradient | 渐变器（<br>线性的渐变<br>\|径向渐变（辐射渐变）<br>\|梯形渐变）<br>边界的模式是左上角（0,0），右下角（1,1）参数是从 0 到 1，一般为实际的盒模式的坐标。必须是升序排序。<br>例：<br>/* linear gradient from white to green */<br>QTextEdit {<br>background:qlineargradient(x1:0, y1:0, x2:1, y2:1,stop:0 white, stop: 0.4 gray, stop:1 green)<br>}<br>((x1，y2)（x2，y2）线性的两个点）<br>/* linear gradient from white to green */<br>QTextEdit {<br>background:qlineargradient(x1:0, y1:0, x2:1, y2:1,<br>stop:0 white, stop: 0.4 rgba(10, 20, 30, 40),stop:1 rgb(0, 200, 230, 200))（透明度）<br>}<br><br>/* conical gradient from white to green */<br>QTextEdit {<br>background:qconicalgradient(cx:0.5,cy:0.5,angle:30,stop:0 white, stop:1 #00FF00)<br>}<br>/*中心点（0.5,0.5）角度 30*/<br>/* radial gradient from white to green */<br>QTextEdit {<br>background:qradialgradient(cx:0, cy:0, radius:1,fx:0.5, fy:0.5, stop:0 white, stop:1 green)<br>}<br>/*中心点（0，0），半径 1 焦点（0.5,0.5）从焦点开始渐变*/ |
| Icon | (Url<br>(disabled<br>\| active<br>\|normal<br>\| selected)<br>? | 一组图标地址 +<br>按钮的（禁用\|活动\|正常\|选择） ？（on\|off）<br>意思是前面括号里四个中的一个状态和后面两个中的一个的组合来控制图片的显示哪一个。<br>例：<br>* { |

www.docin.com

| | | |
|---|---|---|
| | (on \| off)?<br>)* | file-icon: url(file.png),<br>url(file_selected.png)<br>selected;<br>}<br>QMessageBox {<br>dialogbuttonbox-buttons-have-icons:<br>true;<br>dialog-ok-icon: url(ok.svg);<br>dialog-cancel-icon:<br>url(cancel.png), url(grayed_cancel.png)<br>disabled;<br>} |
| Length | Number(px \| pt \| em<br>\| ex)? | 长度单位（数字+其中一个） |
| Number | A decimal<br>integer or a real<br>number | 一些十进制的整数或者是真假值，数字如：0 ，18，<br>+127，-255，12.34，-.5，<br>0008 等。 |
| Origin | margin<br>\| border<br>\| padding<br>\| content | margin: 最外边的矩形. 可以控制两个部件之间的空隙。<br>border: 为边界预留的空间。.<br>padding: 在 border 里面控制 border 与部件内容的空间。<br>content: 部件窗口内容。<br>可以参考 The Box Model. |
| PaletteRole | alternate-base<br>\| base<br>\| bright-text<br>\| button<br>\| button-text<br>\| dark<br>\| highlight<br>\| highlighted-text<br>\| light<br>\| link<br>\| link-visited<br>\| mid<br>\| midlight<br>\| shadow<br>\| text<br>\| window<br>\| window-text | 例子：<br>QPushButton { color: palette(dark); } |
| Radius | Length{1, 2} | 角的弧度<br>1.可以是一个值这样就是角的直接弧度。<br>2.两个值的时候，第一个是横向的弧度，第二个是纵 |

www.docin.com

| | | |
|---|---|---|
| | | 向的弧度。可以制作出一个类似椭圆四分之一圆的角度。 |
| Repeat | repeat-x<br>\| repeat-y<br>\| repeat<br>\| no-repeat | repeat-x: 横向重复图.<br>repeat-y: 纵想重复.<br>repeat: 横纵同时.<br>no-repeat: 不重复. |
| Url | url(filename) | 地址 |

## 六、附属例题解释：惯用方法

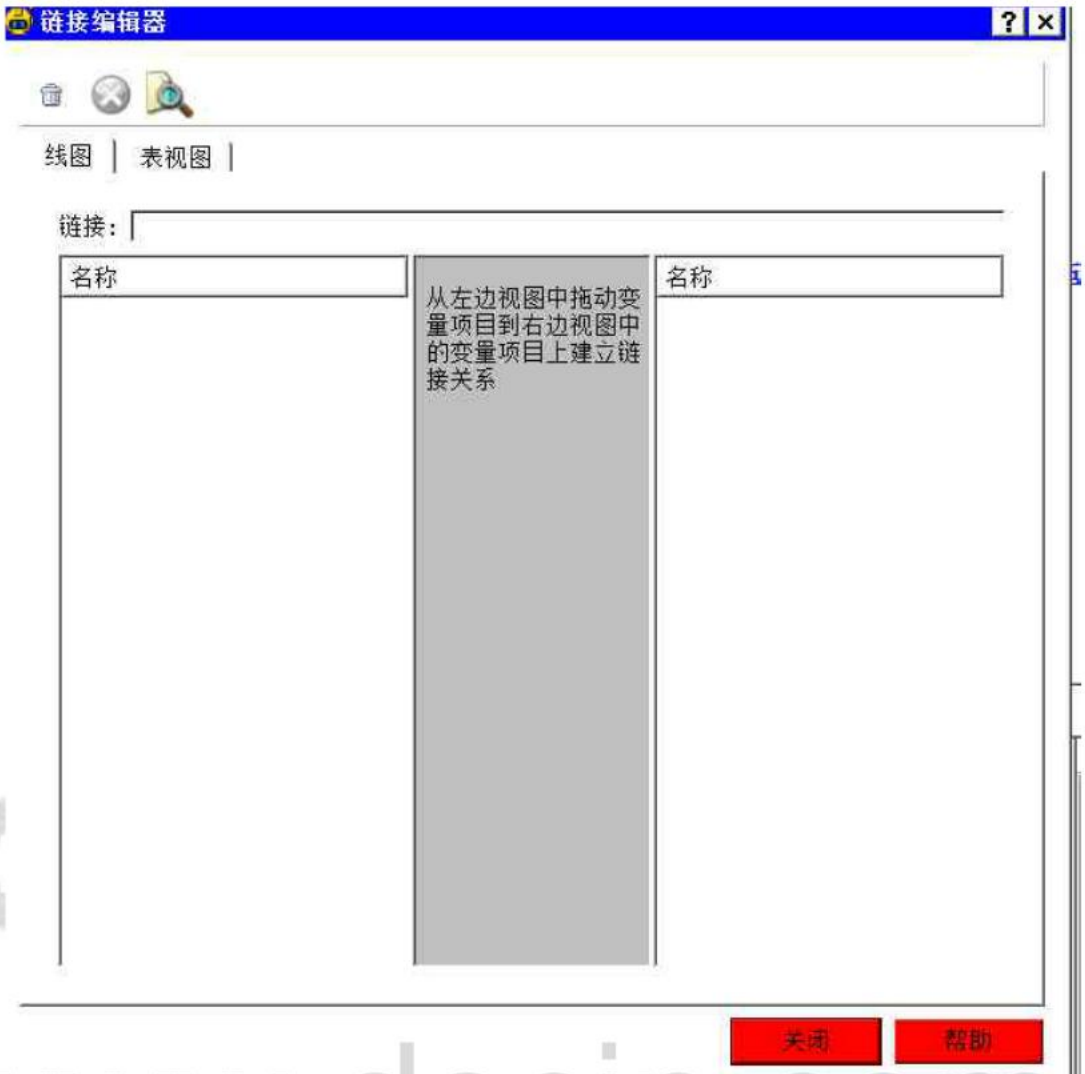
### 1. 定制全局颜色值

```
*{
background:blue;
color:red;
}
```

### 2. 定制按钮(QPushButton)

```
1. QPushButton{
    background-color:red;
    }
    /*imagesForExample:  example_for_QPushButton.png*/
```

www.docin.com




www.docin.com

2. QPushButton{
   background-color:red;
   border-style: outset;
   border-width:2px;
   border-color:beige;
   }/*imagesForExample： example_for_QPushButton2.png*/




www.docin.com

3.
```
QPushButton {
    background-color: red;
    border-style: outset;
    border-width: 2px;
    border-radius: 10px;
    border-color: beige;
    font: bold 14px;
    min-width: 10em;
    padding: 6px;
}
/*imagesForExample：  example_for_QPushButton3.png*/
```




www.docin.com

4.QPushButton {
    background-color: red;
    border-style: outset;
    border-width: 2px;
    border-radius: 10px; /*边缘线的半径*/
    border-color: #fcaf17; /*按钮的边缘线为黄色*/
    font: bold 14px;   /*字体*/
    min-width: 10em;
    padding: 6px;
}

5. QPushButton:pressed {
    background-color: #7fb80e; /*当按钮被按下(pressed)的时候就出现若绿色按钮*/
    border-style: inset;
}

## 3.定制按钮(QPushButton)的 menu indicator 辅助控制器

```
QPushButton::menu-indicator {
    image: url(myindicator.png);
    subcontrol-position: right center;
    subcontrol-origin: padding;
    left: -2px;
}
```

## 4.指定某一个窗口部件进行设置属性

```
QPushButton#pushButton{
    corlor:red;
}
/*指定的按钮 pushButton 前景色为红色*/
```

## 5. 渐变器

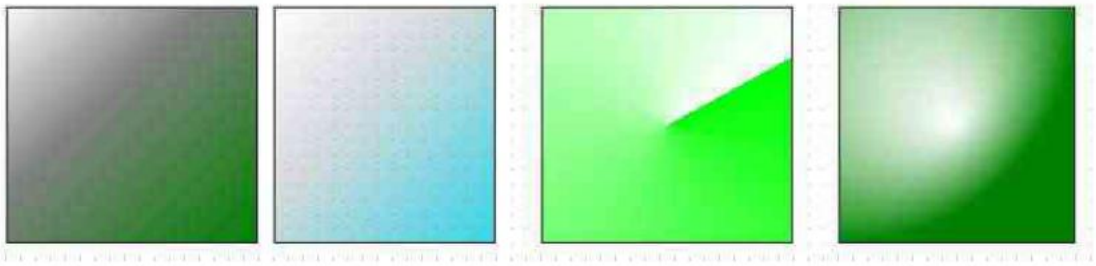
```
QTextEdit {
    background: qlineargradient(x1:0, y1:0, x2:1, y2:1,
                stop:0 white, stop: 0.4 gray, stop:1 green)
}

/* linear gradient from white to green */
QTextEdit {
    background: qlineargradient(x1:0, y1:0, x2:1, y2:1,
                stop:0 white, stop: 0.4 rgba(10, 20, 30, 40), stop:1 rgb(0, 200, 230, 200))
}

/* conical gradient from white to green */
QTextEdit {
    background: qconicalgradient(cx:0.5, cy:0.5, angle:30,
                stop:0 white, stop:1 #00FF00)
}

/* radial gradient from white to green */
QTextEdit {
    background: qradialgradient(cx:0, cy:0, radius: 1,
                fx:0.5, fy:0.5, stop:0 white, stop:1 green)
}
```

www.docin.com



线性渐变　　线性渐变有透明度值　　梯形渐变　　辐射渐变

## 6. 定制 QAbstractScrollArea

```
1. QTextEdit, QListView {
      background-color: #7fb80e;
      background-image: url(images/myindicator.png);
      background-attachment: scroll;
   }

2. QTextEdit, QListView {
      background-color: red;
      background-image: url(images/myindicator.png);
      background-attachment: fixed;
   }
```

/*1 与 2 的区别就在于背景图片是否依赖于滚动条，1.scroll 是表示图片跟随滚动条一起滑动，2.fixed 表示背景图片是不跟随滚动条滑动的。*/

## 7. 定制 QCheckBox

QCheckBox 是与 QRadioButton 的样式是同步的，都是三态部件(1.选中。2.半选中[表示是一个 checkbox 中含一组 checkbox,这一组有的被选中有的不被选中,表现出的这种状态]。3.未选中)

unchecked:表示这个 QCheckBox 没有选中的时候对其设置效果。

Checked：表示是在选中的时候 QCheckBox 对其设置效果。

```
1. QCheckBox {
     spacing: 5px;
  }
/*可点击的小窗口和描述文字之间的空间为 5px*/

2. QCheckBox::indicator {
     width: 13px;
     height: 13px;
  }
/*可选择小窗口的宽高设定*/
```

www.docin.com

```
3. QCheckBox::indicator:unchecked {
     image: url(styles/start.png);
  }
/*可点击的小窗口在不点击时添加的图片*/
/*imagesForExample：example_for_QCheckBox001.png*/

4. QCheckBox::indicator:unchecked:hover {
     image: url(styles/start.png);
  }
/*鼠标划过 QCheckBox 时显示图片*/

5. QCheckBox::indicator:unchecked:pressed {
     image: url(styles/start.png);
  }
/*单击文本描述的时候显示图片，但是小窗口显示的是”对号”符号*/

6. QCheckBox::indicator:checked {
     image: url(styles/start.png);
  }
/*当已被选中的时候单击小窗口和文本描述的时候，小窗口变成这个图片*/

7. QCheckBox::indicator:checked:hover {
     image: url(styles/start.png);
  }
/*当已被选中的时候鼠标划过 QCheckBox 时显示图片*/

8. QCheckBox::indicator:checked:pressed {
     image: url(styles/start.png);
  }
/*当已被选中的时候鼠标点击 QCheckBox 显示图片，但是小窗口显示的是”对号”符号*/

9. QCheckBox::indicator:indeterminate:hover {
     image: url(styles/start.png);
 }
/*不确定状态情况下的鼠标滑过（半选中状态）*/

10. QCheckBox::indicator:indeterminate:pressed {
     image: url(styles/start.png);
 }
/*不确定状态情况下的鼠标滑过（半选中状态）*/
```

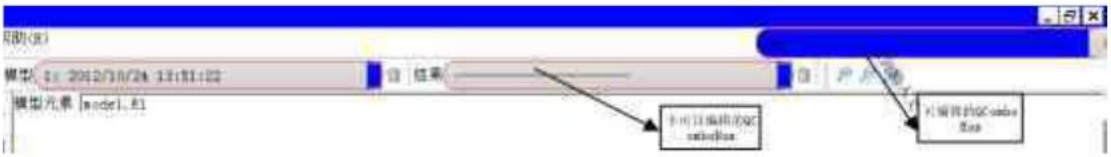
## 8. 定制 QComboBox

```
1. QComboBox {
```

docin 豆丁
www.docin.com

```
    border: 1px solid gray;  /*外围框的 1px 的 gray 色的实线*/
    border-radius: 3px;  /*四个角半径的弧度 3px*/
    padding: 5px 18px 5px 10px;/*中间文本与上、右、下、左之间的空隙*/
    min-width: 20em;   /*QComboBox 最小宽度*/
   }
```

2. 
```
QComboBox:editable {
    background: blue;
   }
   /*可编辑的变为 blue*/
   /*example_for_QComboBox001.png*/
```



3. 
```
QComboBox:!editable, QComboBox::drop-down:editable {
     background: qlineargradient(x1: 0, y1: 0, x2: 0, y2: 1,
                                 stop: 0 #f173ac, stop: 0.4 #00ae9d,
                                 stop: 0.5 #596032, stop: 1.0 #ef4136);
   }
  /*上到下的一种线性渐变器*/
  /*imagesForExample : example_for_QComboBox002.png*/
```

4. 
```
/* QComboBox gets the "on" state when the popup is open */
 /*不可编辑的 QComboBox 和 可编辑的 QComboBox 设置属性为 on*/
 QComboBox:!editable:on, QComboBox::drop-down:editable:on {
    background: qlineargradient(x1: 0, y1: 0, x2: 0, y2: 1,
                                stop: 0 #f173ac, stop: 0.4 #00ae9d,
                                stop: 0.5 #596032, stop: 1.0 #ef4136)
   }
  /*当点击展开按钮时的时候才可以出现*/
  /*imagesForExample :  example_for_QComboBox002.png 现象*/
```



5. 
```
QComboBox:on { /* shift the text when the popup opens */
    padding-top: 3px;
    padding-left: 4px;
  }
/*当点击展开的按钮时（on），文本与外框间的空隙统一设置 top 3px ， left 4px*/
```

www.docin.com

```
6. QComboBox::drop-down {
     subcontrol-origin: padding;
     subcontrol-position: top right;
     width: 15px;

     border-left-width: 1px;
     border-left-color: darkgray;
     border-left-style: solid; /* 一条实线 */
     border-top-right-radius: 3px; /* 不确定状态 */
     border-bottom-right-radius: 3px;
   }

7. QComboBox::down-arrow {
     image: url(styles/start.png);
   }
  /*更换下拉按钮的向下的图标*/

8. QComboBox::down-arrow:on { /* shift the arrow when popup is open */
     top: 5px;
     left: 1px;
  }
```

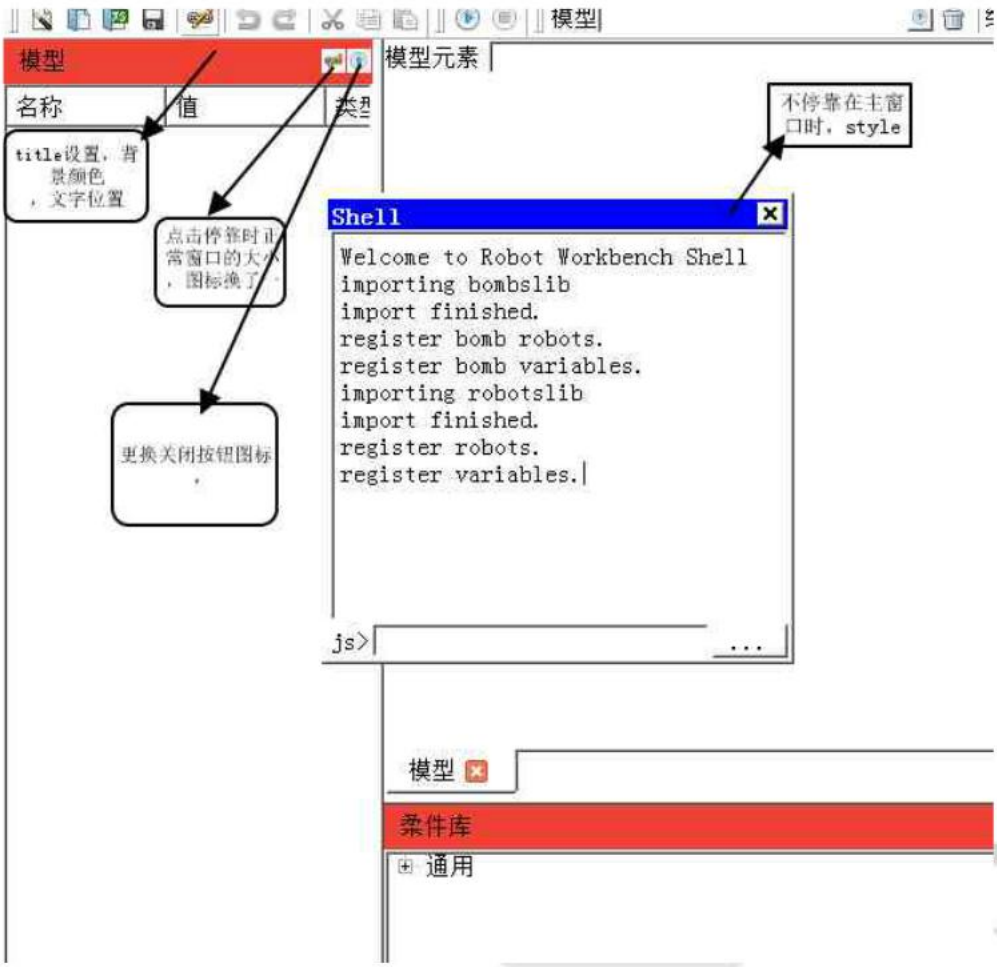
## 9. 定制 QDockWidget

QDockWidget 的标题栏和按钮是可以自定制的。如下例题：

```
1.  QDockWidget {
     border: 5px solid #ef4136;     /*设置浮动窗口(QDockWidget)外框线*/
     titlebar-close-icon: url( styles/start.png);/*设置浮动窗口的关闭按钮*/
     titlebar-normal-icon: url( styles/linkedit.png);/*设置浮动窗口正常大小的按钮图标*/
   }
/*imagesForExample :  example_for_QDockWidget001.png 现象*/


2. QDockWidget::title {
     text-align: left; /* 浮动窗口的名字在左侧 */
     background: #ef4136;   /*标题背景色*/
     padding-left: 10px;   /*浮动窗口的名字与标题栏左侧外框距离*/
 }
/*imagesForExample :  example_for_QDockWidget001.png 现象*/
```

www.docin.com




3. QDockWidget::close-button, QDockWidget::float-button {

border: 1px solid transparent; /*两个按钮的外框线 1px，实线，transparent（透明的)*/

background: darkgray;　/*按钮的背景颜色*/

padding: 0px;　/*两个按钮的里面的图标与图标外框的距离*/

}

4. QDockWidget::close-button:hover, QDockWidget::float-button:hover {

background: gray;

border: 1px solid transparent;

}

/*单独设置背景颜色时需要设置 border 颜色值才可以*/

5. QDockWidget::close-button:pressed, QDockWidget::float-button:pressed {

padding: 1px -1px -1px 1px;

border: 1px solid transparent;

}

/*同理也要设置 border 的值才可以出现效果*/

- 如果你想把 QDockWidget 的两个按钮放在 left 例子 6、7 如下：

1. QDockWidget {

border: 1px solid lightgray;

```
    titlebar-close-icon: url(styles/start.png);
    titlebar-normal-icon: url(styles/linkedit.png);
}

2.QDockWidget::title {
    text-align: left;
    background: lightgray;
    padding-left: 35px;
}

3.QDockWidget::close-button, QDockWidget::float-button {
    background: darkgray;
    padding: 0px;
    icon-size: 14px; /* maximum icon size 下面定义了 icon 的大小*/
}

4.QDockWidget::close-button:hover, QDockWidget::float-button:hover {
    background: gray;
}

5.QDockWidget::close-button:pressed, QDockWidget::float-button:pressed {
    padding: 1px -1px -1px 1px;
}

6.QDockWidget::close-button {
    subcontrol-position: top left;
    subcontrol-origin: margin;
    position: absolute;
    top: 0px; left: 0px; bottom: 0px;
    width: 14px;
}
/*上面是关闭按钮的设置*/

7.QDockWidget::float-button {
    subcontrol-position: top left;
    subcontrol-origin:margin;  /*盒模型的最外面的那个控件选择默认是padding的，
此处设置为 margin*/
    position: absolute;  /*绝对位置（它是相对于关闭按钮存在的）*/
    top: 0px; left: 16px; bottom: 0px;
    width: 14px;
}
/*上面是浮动按钮的设置*/
```

www.docin.com

## 10. 定制 QFrame

一般用盒模式来设置 QFrome 的样式

```
1.QFrame, QLabel, QToolTip {
     border: 2px solid green;
     border-radius: 4px;
     padding: 2px;
     background-image: url(styles/start.png);
 }
/*这个外框是 2px 实线，绿色，圆角半径是 4px 的，盒模型的 padding 是 2px，背景图片*/
```

## 11. 定制 QGroupBox

把 QGroupBox 的标题移到中间

```
QGroupBox {
     background-color: qlineargradient(x1: 0, y1: 0, x2: 0, y2: 1,
                                       stop: 0 #E0E0E0, stop: 1 #FFFFFF);
     border: 2px solid gray;
     border-radius: 5px;
     margin-top: 1ex; /* leave space at the top for the title */
 }

 QGroupBox::title {
     subcontrol-origin: margin;
     subcontrol-position: top center; /* position at the top center */
     padding: 0 3px;
     background-color: qlineargradient(x1: 0, y1: 0, x2: 0, y2: 1,
                                       stop: 0 #ed1941, stop: 1 #1d953f);
 }
/*imagesForExample :  example_for_QGroupBox001.png*/
```



可以像 QCheckBox 一样设置它的样式。

```
QGroupBox::indicator {
     width: 13px;
     height: 13px;
 }

 QGroupBox::indicator:unchecked {
     image: url(styles/start.png);
 }
```

## 12. 定制 QHeaderView

```
QHeaderView::section {
     background-color: qlineargradient(x1:0, y1:0, x2:0, y2:1,
                                       stop:0 #616161, stop: 0.5 #505050,
                                       stop: 0.6 #434343, stop:1 #656565);
    /*标题头的渐变色*/
     color: #ef4136; /*字体的颜色*/
     padding-left: 4px; /*标题名字和左侧的位置*/
     border: 1px solid #6c6c6c;/*外框的线属性*/
 }

 QHeaderView::section:checked
 {
     background-color: red;  /*点击后标题头变为红色*/
 }

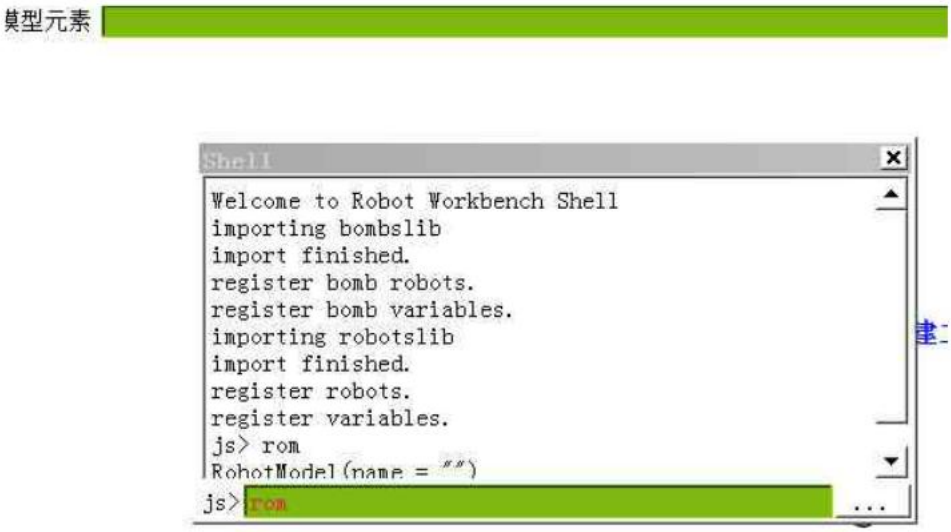
 QHeaderView::down-arrow {
     image: url(styles/start.png);/*图标设置用来排序的*/
 }

 QHeaderView::up-arrow {
     image: url(styles/linkedit.png);
 }
```

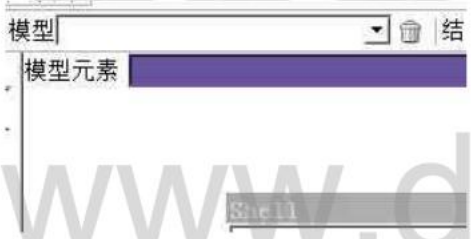
## 13 定制 QLineEdit

```
1.QLineEdit{
    background-color: #7fb80e;
    color: #fcaf17;
  }
/*前景（输入文字）颜色是#fcaf17，背景颜色 #7fb80e;*/
/*imagesForExample :  example_for_QLineEdit001.png*/
```

docin 豆丁 www.docin.com




2.QLineEdit[readOnly="true"]{
    color: #ea66a6;
    background-color: #6950a1;
 }
/*只读(只可以读取不可以输入文字进行编辑的文字)对话框前景（文字）颜色是#ea66a6，背景颜色 #6950a1*/
/*imagesForExample :  example_for_QLineEdit003.png*/




www.docin.com

3.QLineEdit { color: red; }
  QLineEdit[readOnly="true"] { color: #fcaf17;  }
  QDialog QLineEdit { background-color: #7fb80e; }
/*在 QDialog 上的 QLineEdit 的背景颜色都为 #7fb80e*/
/*imagesForExample :  example_for_QLineEdit002.png*/



## 14. 定制 QListView

设置表格交替的背景颜色可以这样设置。

```
QListView {
        alternate-background-color: yellow;
    }
```

当鼠标划过表格的某一行的时候，我们可以如下的设置。

```
QListView {
    show-decoration-selected: 1; /* make the selection span the entire width ofthe
view */
    }

QListView::item:alternate {
    background: #EEEEEE;
    }

QListView::item:selected {
    border: 1px solid #6a6ea9;
    }
    /*选择时这一行被#6a6ea91px 的实线包围*/

QListView::item:selected:!active {
     background: qlineargradient(x1: 0, y1: 0, x2: 0, y2: 1,
                                 stop: 0 #ABAFE5, stop: 1 #8588B2);
    }
    /* 当点击一次表格，之后离开，单击后的这一行的颜色*/

QListView::item:selected:active {
    background: qlineargradient(x1: 0, y1: 0, x2: 0, y2: 1,
                                stop: 0 #6a6ea9, stop: 1 #888dd9);
    }
    /*鼠标点击这一行时的背景颜色*/

QListView::item:hover {
    background: qlineargradient(x1: 0, y1: 0, x2: 0, y2: 1,
                                stop: 0 #FAFBFE, stop: 1 #DCDEF1);
  }
  /*鼠标划过时背景颜色变化*/
```

## 15. 定制 QMainWindow

```
QMainWindow::separator {
    background: yellow;
```

docin 豆丁 www.docin.com

```
    width: 10px; /* when vertical */
    height: 10px; /* when horizontal */
}
/*主窗口的分割线为 yellow*/

QMainWindow::separator:hover {
    background: red;
}
/*当鼠标划过时为 red*/
```

## 16. 定制 QMenu

```
1. QMenu {
    background-color: #ABABAB; /* sets background of the menu */
    border: 1px solid black;
}
/*QMenu 的背景颜色，这个 QMenu 的框为 1px 实线 的 black */
2. QMenu::item {
    /* sets background of menu item. set this to something non-transparent
        if you want menu color and menu item color to be different */
    background-color: transparent;
}
/*透明度*/

3. QMenu::item:selected { /* when user selects item using mouse or keyboard */
    background-color: #102b6a;
}
/*选择 item 时的背景颜色*/

4. QMenu { /* 菜单 */
    background-color: white;
    margin: 0px; /* some spacing around the menu */
    border: 1px solid green;
}

QMenu::item {
    padding: 2px 25px 2px 20px;
    border: 1px solid transparent; /* reserve space for selection border */
}

QMenu::item:selected {
    border-color: darkblue;
    background: rgba(100, 100, 100, 150);
}
```

www.docin.com

```
QMenu::icon:checked { /* appearance of a 'checked' icon */
        background: red;
        border: 1px inset gray;
        position: absolute;
        top: 1px;
        right: 1px;
        bottom: 1px;
        left: 1px;
}

QMenu::separator {
        height: 2px;
        background: lightblue;
        margin-left: 10px;
        margin-right: 5px;
 }

QMenu::indicator {
        width: 13px;
        height: 13px;
}

 /* non-exclusive indicator = check box style indicator (see QActionGroup::setExclusive) */
QMenu::indicator:non-exclusive:unchecked {
        /*image: url(styles/start.png);*/
        /*image: url(styles/start.png);*/
}

QMenu::indicator:non-exclusive:unchecked:selected {
        /*image: url(styles/start.png);*/
        image: url(styles/start.png);
}

QMenu::indicator:non-exclusive:checked {
        /*image: url(styles/start.png);*/
        image: url(styles/start.png);
}

QMenu::indicator:non-exclusive:checked:selected {
        /*image: url(styles/start.png);*/
        image: url(styles/start.png);
}
```

docin 豆丁
www.docin.com

```
/* exclusive indicator = radio button style indicator (see QActionGroup::setExclusive) */
QMenu::indicator:exclusive:unchecked {
        image: url(styles/start.png);
}

QMenu::indicator:exclusive:unchecked:selected {
        image: url(styles/start.png);
}

QMenu::indicator:exclusive:checked {
        image: url(styles/start.png);
}

QMenu::indicator:exclusive:checked:selected {
        image: url(styles/start.png));
}
```

## 17. 定制 QMenuBar

我们定制菜单栏时的用法，还有很多的用法多找找表格

```
 QMenuBar { /*  菜单栏  */
     background-color: qlineargradient(x1:0, y1:0, x2:0, y2:1,
     stop:0 lightgray, stop:1 darkgray);
 }

QMenuBar::item {
        spacing: 3px; /* spacing between menu bar items */
        padding: 3px 4px;
        background: transparent;
        border-radius: 4px;
}

QMenuBar::item:selected { /* when selected using mouse or keyboard */
        background: #a8a8a8;
}
/*选择时的背景颜色*/

QMenuBar::item:pressed {
        background: #1d953f;
}
/*点击鼠标时菜单栏的背景颜色*/
```

docin 豆丁
www.docin.com

## 18. 定制 QProgressBar

```
QProgressBar {
        border: 2px solid grey;
        border-radius: 5px;
 }
/*外框线是 2px*/

 QProgressBar::chunk {
        background-color: #05B8CC;
        width: 20px;
 }
/*背景颜色，和宽度*/

QProgressBar {
        border: 2px solid grey;
        border-radius: 5px;
        text-align: center;
 }
/*文本在中间，圆角角度*/

QProgressBar::chunk {
        background-color: #CD96CD;
        width: 10px;
        margin: 0.5px;
 }
/*结果就是一块一块的增加，间隔是 0.5px*/
```

docin 豆丁
www.docin.com

## 19. 定制 QPushButton

```
QPushButton {
     border: 2px solid #8f8f91;
     border-radius: 6px;
     background-color: qlineargradient(x1: 0, y1: 0, x2: 0, y2: 1,
                                       stop: 0 #d71345, stop: 1 #dadbde);
     min-width: 80px;
 }
/*按钮的 border 为 2px 实线  颜色：#8f8f91，半径为 6px，渐变器，最小宽度*/

 QPushButton:pressed {
     background-color: qlineargradient(x1: 0, y1: 0, x2: 0, y2: 1,
                                       stop: 0 #7fb80e, stop: 1 #f6f7fa);
 }
```

```
 QPushButton:flat {
     border: none; /* no border for a flat push button */
 }

 QPushButton:default {
     border-color: #4a3113; /* make the default button prominent */
}
/*imagesFoeExample: example_for_QPushButton001.png*/
```



可以用:checked 状态来设置 QPushButton。
当把按钮放入菜单栏上时我们可以这样设置它的属性。

```
QPushButton:default {
     border-color: #4a3113; /* make the default button prominent */
 }

QPushButton:open { /* when the button has its menu open */
     background-color: qlineargradient(x1: 0, y1: 0, x2: 0, y2: 1,
                                       stop: 0 #1d953f, stop: 1 #f6f7fa);
 }

QPushButton::menu-indicator {
     image: url(menu_indicator.png);
     subcontrol-origin: padding;
     subcontrol-position: bottom right;
 }

QPushButton::menu-indicator:pressed, QPushButton::menu-indicator:open {
     position: relative;
```

www.docin.com

```
    top: 2px; left: 2px; /* shift the arrow by 2 px */
}
```

## 20. 定制 QRadioButton

```
 QRadioButton::indicator {
    width: 13px;
    height: 13px;
}
/*定制可选择的小圆圈的大小*/
QRadioButton::indicator::unchecked {
    image: url(:/images/radiobutton_unchecked.png);
}

QRadioButton::indicator:unchecked:hover {
    image: url(:/images/radiobutton_unchecked_hover.png);
}

QRadioButton::indicator:unchecked:pressed {
    image: url(:/images/radiobutton_unchecked_pressed.png);
}

QRadioButton::indicator::checked {
    image: url(:/images/radiobutton_checked.png);
}

QRadioButton::indicator:checked:hover {
    image: url(:/images/radiobutton_checked_hover.png);
}

QRadioButton::indicator:checked:pressed {
    image: url(:/images/radiobutton_checked_pressed.png);
}
/*小圆圈被选中和没被选中时的鼠标状态*/
```

## 21. 定制 QScrollBar

QScrollBar 这个滚动条可是通过下面的方式设置，也可用 handle，add-line，sub-line 一些辅助控制器设置。

Note:如果它定义了一个属性或者是副属性,那么我们也要定义它的子部件的属性和副属性。

```
QScrollBar:horizontal {
    border: 2px solid grey;
```

www.docin.com

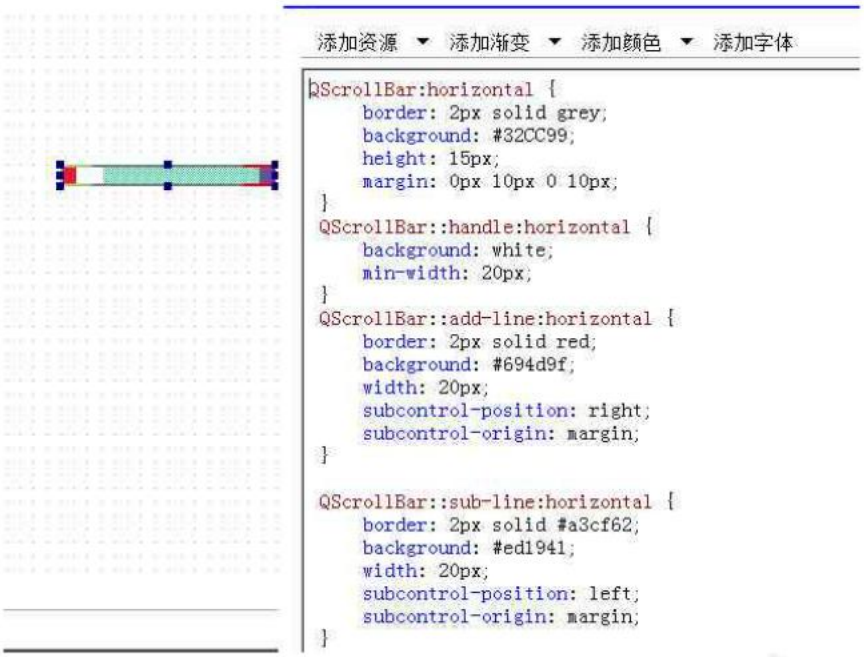
```
    background: #32CC99;
    height: 15px;
    margin: 0px 10px 0 10px;/*两头的是 10px 的小方块*/
}
/*横向的滚动条外边缘线 2px 实线 grey 背景颜色是#32CC99, 高为 15px*/

QScrollBar::handle:horizontal {
    background: white;
    min-width: 20px;
}
/*滚动条的滑块（handle）为白色，宽度为 20px*/

QScrollBar::add-line:horizontal {
    border: 2px solid grey;
    background: #694d9f;
    width: 20px;
    subcontrol-position: right;
    subcontrol-origin: margin;
}
/*向添加内容方向的小方块颜色，宽度，位置，边缘线*/

QScrollBar::sub-line:horizontal {
    border: 2px solid grey;
    background: #ed1941;
    width: 20px;
    subcontrol-position: left;
    subcontrol-origin: margin;
}
/*向减少内容方向的小方块颜色，宽度，位置，边缘线*/
/*imagesForExample: example_for_QScrollBar001.png*/
```

docin 豆丁
www.docin.com




对于箭头方向的设置 left-arrow 和 right-arrow

```
QScrollBar:left-arrow:horizontal, QScrollBar::right-arrow:horizontal {
     border: 2px solid grey;
     width: 3px;
     height: 3px;
     background: white;
 }

 QScrollBar::add-page:horizontal, QScrollBar::sub-page:horizontal {
     background: none;
 }
/*定制一个箭头的大小*/
```

这个是设置一个横向的加内容和减内容的的小方块放到了一起。

```
QScrollBar:horizontal {
     border: 2px solid green;
     background: cyan;
     height: 15px;
     margin: 0px 40px 0 0px;
 }

 QScrollBar::handle:horizontal {
     background: gray;
     min-width: 20px;
 }

 QScrollBar::add-line:horizontal {
```

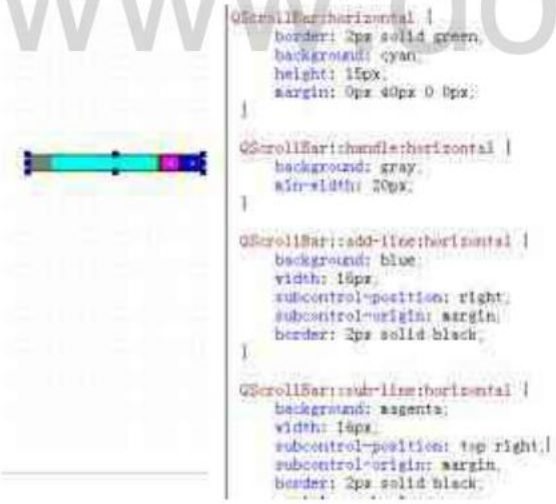
www.docin.com

```
    background: blue;
    width: 16px;
    subcontrol-position: right;
    subcontrol-origin: margin;
    border: 2px solid black;
}

QScrollBar::sub-line:horizontal {
    background: magenta;
    width: 16px;
    subcontrol-position: top right;
    subcontrol-origin: margin;
    border: 2px solid black;
    position: absolute;
    right: 20px;
}

QScrollBar:left-arrow:horizontal, QScrollBar::right-arrow:horizontal {
    width: 3px;
    height: 3px;
    background: pink;
}

QScrollBar::add-page:horizontal, QScrollBar::sub-page:horizontal {
    background: none;
}
/*imagesFoeExample: example_for_QScrollBar002.png*/
```



竖向的滚动条和纵向的滚动条设置时类似的。

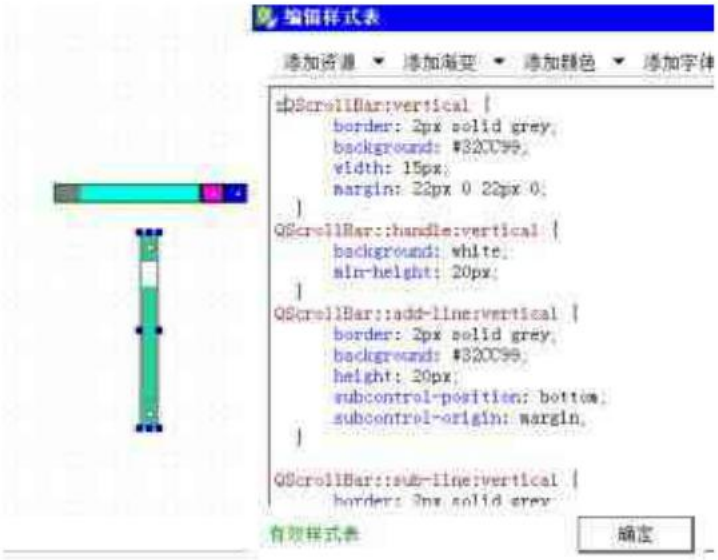
```
QScrollBar:vertical {
      border: 2px solid grey;
      background: #32CC99;
```

```
    width: 15px;
    margin: 22px 0 22px 0;
}
QScrollBar::handle:vertical {
    background: white;
    min-height: 20px;
}
QScrollBar::add-line:vertical {
    border: 2px solid grey;
    background: #32CC99;
    height: 20px;
    subcontrol-position: bottom;
    subcontrol-origin: margin;
}

QScrollBar::sub-line:vertical {
    border: 2px solid grey;
    background: #32CC99;
    height: 20px;
    subcontrol-position: top;
    subcontrol-origin: margin;
}
QScrollBar::up-arrow:vertical, QScrollBar::down-arrow:vertical {
    border: 2px solid grey;
    width: 3px;
    height: 3px;
    background: white;
}

QScrollBar::add-page:vertical, QScrollBar::sub-page:vertical {
    background: none;
}
/*imagesForExample: example_for_QScrollBar003.png*/
```

docin 豆丁
www.docin.com




## 22. 定制 QSizeGrip

仅仅是设置一张图片

```
QSizeGrip {
    image: url(styles/start.png));
    width: 16px;
    height: 16px;
 }
```

## 23. 定制 QSlider

定制一个横向的窗口分割器

```
QSlider::groove:horizontal {
    border: 1px solid #ed1941;
    height: 8px;
/* the groove expands to the size of the slider by default. by giving it a height, it has a fixed size */
    background: qlineargradient(x1:0, y1:0, x2:0, y2:1, stop:0 #494e8f, stop:1 #c4c4c4);
    margin: 2px 0;
 }
/*QSlider 的槽部分设置*/

 QSlider::handle:horizontal {
    background: qlineargradient(x1:0, y1:0, x2:1, y2:1, stop:0 #007d65, stop:1 #8f8f8f);
    border: 1px solid #5c5c5c;
    width: 18px;
```

www.docin.com

```
    margin: -2px 0; /* handle is placed by default on the contents rect of the groove.
Expand outside the groove */
    border-radius: 3px;
 }
/*QSlider 的滑块设置*/
/* imagesForExample: example_for_QSpliter001.png*/
```

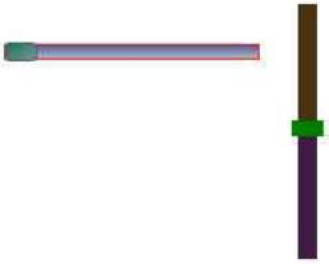



```
QSlider::groove:vertical {
    background: red;
    position: absolute; /* absolutely position 4px from the left and right of the
widget. setting margins on the widget should work too... */
    left: 4px; right: 4px;
}
 QSlider::handle:vertical {
    height: 10px;
    background: green;
    margin: 0 -4px; /* expand outside the groove */
 }

 QSlider::add-page:vertical {
    background: white;
 }

 QSlider::sub-page:vertical {
    background: pink;
 }
/* imagesForExample: example_for_QSpliter002.png*/
```

www.docin.com



## 24. 定制 QSpinBox

```
QSpinBox {
    padding-right: 15px; /* make room for the arrows */
    border-image: url(styles/start.png) 4;
    border-width: 3;
}

QSpinBox::up-button {
    subcontrol-origin: border;
    subcontrol-position: top right; /* position at the top right corner */

    width: 16px; /* 16 + 2*1px border-width = 15px padding + 3px parent border */
    border-image: url(styles/start.png) 1;
    border-width: 1px;
}

QSpinBox::up-button:hover {
    border-image: url(styles/start.png) 1;
}

QSpinBox::up-button:pressed {
    border-image: url(styles/start.png) 1;
}

QSpinBox::up-arrow {
    image: url(styles/start.png);
    width: 7px;
    height: 7px;
}

QSpinBox::up-arrow:disabled, QSpinBox::up-arrow:off { /* off state when value is max */
```

docin 豆丁
www.docin.com

```
    image: url(styles/start.png;
}

QSpinBox::down-button {
    subcontrol-origin: border;
    subcontrol-position: bottom right; /* position at bottom right corner */

    width: 16px;
    border-image: url(styles/start.png) 1;
    border-width: 1px;
    border-top-width: 0;
}

QSpinBox::down-button:hover {
    border-image: url(styles/start.png 1;
}

QSpinBox::down-button:pressed {
    border-image: url(styles/start.png) 1;
}

QSpinBox::down-arrow {
    image: url(styles/start.png);
    width: 7px;
    height: 7px;
}

QSpinBox::down-arrow:disabled,
QSpinBox::down-arrow:off { /* off state when value in min */
    image: url(styles/start.png);
}
```

## 25. 定制 QSplitter

```
QSplitter::handle {
    image: url(styles/start.png));
}
/*句柄图片*/
QSplitter::handle:horizontal {
    width: 2px;
}

QSplitter::handle:vertical {
    height: 2px;
```

docin 豆丁
www.docin.com

```
}

QSplitter::handle:pressed {
    image: url(styles/start.png));
}
```

## 26. 定制 QStatusBar

提供状态栏的背景颜色和状态栏里面 item 的外框线

```
1.  QStatusBar {
        background: brown;
    }

    QStatusBar::item {
        border: 1px solid red;
        border-radius: 3px;
    }
```

部件被加到状态栏上时这样设置才可以生效

```
2.QStatusBar QLabel {
        border: 3px solid white;
    }
```

## 27. 定制 QTabWidget 和 QTabBar

```
QTabWidget::pane { /* The tab widget frame */
    border-top: 2px solid #C2C7CB;
}
/*外框线*/

QTabWidget::tab-bar {
    left: 5px; /* move to the right by 5px */
}
/*tab 页的按钮统一向右移 5px*/

 /* Style the tab using the tab sub-control. Note that it reads QTabBar _not_ QTabWidget */
/*tab 一般用 QTabBar 控制如下*/

QTabBar::tab {
    background: qlineargradient(x1: 0, y1: 0, x2: 0, y2: 1,
                                stop: 0 #E1E1E1, stop: 0.4 #DDDDDD,
                                stop: 0.5 #D8D8D8, stop: 1.0 #D3D3D3);
    border: 2px solid #C4C4C3;
```

docin 豆丁
www.docin.com

```
        border-bottom-color: #C2C7CB; /* tab 的下线， pane 临近的线但是不会和 pane 冲突*/
        border-top-left-radius: 4px;/*左上角的弧度*/
        border-top-right-radius: 4px;
        min-width: 8ex;     /*最小的 tab*/
        padding: 2px; /*可以理解为 tab 的高度*/
    }
/*设置 tab，背景是一个渐变色，外框线 2px 实线 #C4C4C3，*/

    QTabBar::tab:selected, QTabBar::tab:hover {
        background: qlineargradient(x1: 0, y1: 0, x2: 0, y2: 1,
                                    stop: 0 #fafafa, stop: 0.4 #f4f4f4,
                                    stop: 0.5 #e7e7e7, stop: 1.0 #fafafa);
    }
/*鼠标划过和鼠标选择时的背景颜色*/

    QTabBar::tab:selected {
        border-color: #9B9B9B;
        border-bottom-color: #C2C7CB; /* same as pane color */
    }
/*鼠标单击后 tab 的边缘线颜色和下边缘线的颜色*/

    QTabBar::tab:!selected {
        margin-top: 2px; /* make non-selected tabs look smaller */
    }
/*没被选择时 tab 整体下降 2px，目的凸显被选中的 tab*/
/*imagesFoeExamole：  example_for_QTabWidget001.png*/
```




上面的 tab 是酷似一个按钮一个按钮的，但是有时候我们想要一种效果就是，当我们选择 tab 的时候这个 tab 凸现出来，并放大，如下一个例子：

```
QTabWidget::pane { /* The tab widget frame */
        border-top: 2px solid #C2C7CB;
 }
/*tabwidget 的上面线的颜色*/

 QTabWidget::tab-bar {
        left: 5px; /* move to the right by 5px */
 }
/*整体的 tab 键左移 5px*/

 QTabBar::tab {
        background: qlineargradient(x1: 0, y1: 0, x2: 0, y2: 1,
                                    stop: 0 #E1E1E1, stop: 0.4 #DDDDDD,
                                    stop: 0.5 #D8D8D8, stop: 1.0 #D3D3D3);
        border: 2px solid #C4C4C3;
        border-bottom-color: #C2C7CB; /* same as the pane color */
        border-top-left-radius: 4px;
        border-top-right-radius: 4px;
        min-width: 8ex;
        padding: 2px;
 }

 QTabBar::tab:selected, QTabBar::tab:hover {
        background: qlineargradient(x1: 0, y1: 0, x2: 0, y2: 1,
                                    stop: 0 #fafafa, stop: 0.4 #f4f4f4,
                                    stop: 0.5 #e7e7e7, stop: 1.0 #fafafa);
 }

 QTabBar::tab:selected {
        border-color: #9B9B9B;
        border-bottom-color: #C2C7CB; /* same as pane color */
 }
/*选择时的线的颜色*/

 QTabBar::tab:!selected {
        margin-top: 2px; /* make non-selected tabs look smaller */
 }
/*不被选择时下移 2px*/

 /* make use of negative margins for overlapping tabs */
 QTabBar::tab:selected {
        /* expand/overlap to the left and right by 4px */
        margin-left: -4px;
```

docin 豆丁
www.docin.com

```
        margin-right: -4px;
    }
/*当被点击的时候就会向外左右扩展 4px*/

    QTabBar::tab:first:selected {
        margin-left: 0; /* the first selected tab has nothing to overlap with on the left */
    }
/*意思是说你选择第一个 tab 它的左面是不增加的*/

    QTabBar::tab:last:selected {
        margin-right: 0; /* the last selected tab has nothing to overlap with on the right */
    }
/*选择最后一个的时候它的右侧是不增加的*/

    QTabBar::tab:only-one {
        margin: 0; /* if there is only one tab, we don't want overlapping margins */
    }
/*只有一个的时候是不扩展的*/
```

3. 有时候我们想要一个可以把 tab 放在 pane 线上。例子如下：

```
QTabWidget::pane { /* The tab widget frame */
        border-top: 2px solid #C2C7CB;
        position: absolute;
        top: -0.5em;
    }
/*widget 的上线颜色为#C2C7CB，位置是完全绝对与原来的位置，-0.5 是在原来（原来是在
tab 下线的下面）的位置上面 0.5em 的距离。*/

    QTabWidget::tab-bar {
        alignment: center;
    }
/*所有的 tab 都放在中间的位置*/

    QTabBar::tab {
        background: qlineargradient(x1: 0, y1: 0, x2: 0, y2: 1,
                                    stop: 0 #E1E1E1, stop: 0.4 #DDDDDD,
                                    stop: 0.5 #D8D8D8, stop: 1.0 #D3D3D3);
        border: 2px solid #C4C4C3;
        border-bottom-color: #C2C7CB; /* same as the pane color */
        border-top-left-radius: 4px;
        border-top-right-radius: 4px;
        min-width: 8ex;
        padding: 2px;
```

www.docin.com

```
 }
/*对 tab 设置颜色，外框*/
 QTabBar::tab:selected, QTabBar::tab:hover {
      background: qlineargradient(x1: 0, y1: 0, x2: 0, y2: 1,
                                  stop: 0 #fafafa, stop: 0.4 #f4f4f4,
                                  stop: 0.5 #e7e7e7, stop: 1.0 #fafafa);
 }
/*tab 键对鼠标操作的反应*/

 QTabBar::tab:selected {
      border-color: #9B9B9B;
      border-bottom-color: #C2C7CB; /* same as pane color */
 }
```

下面是对 tear 设置的，当 tab 页过多页面有限显示不出所有的 tab，系统一般会自动为我们添加一个 scroller 来向左向右操作 tab 键，以至于你可以操作所有的 tab。以下是对 tab 的 scroller 设置的样式代码：

```
QTabBar::tear {
      image: url(styles/linkedit.png);
 }

 QTabBar::scroller { /* the width of the scroll buttons */
      width: 20px;
 }
/*scroller 是两个按钮的形式，它的宽度为 20px*/

 QTabBar QToolButton { /* the scroll buttons are tool buttons */
      border-image: url(styles/start.png) 2;
      border-width: 2px;
 }
/*给 buttons 添加图片，九宫格分割法分割图片离四个角的距离是 2px*/
/*按钮的外框的线粗*/

 QTabBar QToolButton::right-arrow { /* the arrow mark in the tool buttons */
      image: url(styles/start.png);
 }
/*右按钮*/

 QTabBar QToolButton::left-arrow {
      image: url(styles/start.png);
 }
/*左按钮*/
```

当有同样的设置时比如

```
QTabWidget::tab-bar {
```

www.docin.com

```
        alignment: center;
 }
/*居中*/
QTabWidget::tab-bar {
        left: 5px; /* move to the right by 5px */
 }
/*左起 5px*/同时设置就会以 left：5px 为准*/
```

## 28. 定制 QTableView

这是一个表格，当你选择某一个表格的时候，你会设置这个表格的颜色，如下：

```
QTableView {
        selection-background-color: qlineargradient(x1: 0, y1: 0, x2: 0.5, y2: 0.5,
                                                stop: 0 #FF92BB, stop: 1 white);
 }
/*selection-background-color 选择的背景颜色*/

QTableView QTableCornerButton::section {
        background: red;
        border: 2px outset red;
 }
/*表头的重合的部分的颜色变化*/
```

## 29. 定制 QToolBar

工具栏的设定，

```
QToolBar {
        background: red;
        spacing: 3px; /* spacing between items in the tool bar */
 }
/*工具栏的背景颜色*/

 QToolBar::handle {
        image: url(styles/start.png);
 }
 /*工具栏可移动的 handle 的图标*/
```

## 30. 定制 QToolBox

这是如同 QQ 好友类型的部件。如下例子：

```
QToolBox::tab {
        background: qlineargradient(x1: 0, y1: 0, x2: 0, y2: 1,
```

www.docin.com

```
                                        stop: 0 #E1E1E1, stop: 0.4 #DDDDDD,
                                        stop: 0.5 #7fb80e, stop: 1.0 #D3D3D3);
    border-radius: 5px;
    color: darkgray;
 }
/*按钮的背景色是渐变的，角弧度 5px，字体的颜色*/

 QToolBox::tab:selected { /* italicize selected tabs */
    font: italic;
    color: white;
 }
/*选中一个的时候它的字体变化了，字的颜色变为 white*/
```

## 31. 定制 QToolButton

ToolButton 有三种形式的：
1.当 QToolButton 没有在 menu 上时他可以像定制，QPushButton 一样定制它；
2.当 QToolButton 在 menu 上，并且他的 QToolButton：： popupMode 设置为 QToolButton：： DelayedPopup 或者 QToopBUtton::instantPopup,就可以像定制 QPushButton 在菜单栏里用 menu-indicator 定制样式；
3.当 QToolButton 的 QToolButton：： popupMode 设置为 QToolButton：： MenuButtonPopup,在这种的情况下例子如下：

```
QToolButton { /* all types of tool button */
    border: 2px solid #8f8f91;
    border-radius: 6px;
    background-color: qlineargradient(x1: 0, y1: 0, x2: 0, y2: 1,
                                      stop: 0 #f6f7fa, stop: 1 #dadbde);
 }
/*所有的 toolbutton 的外框颜色，背景颜色，和角弧度*/

 QToolButton[popupMode="1"] { /* only for MenuButtonPopup */
    padding-right: 20px; /* make way for the popup button */
 }
/*当设置为 popupMode 的性质为真的时候，（只对 MenuButtonPopup 按钮有效）他的右侧填充为 20px*/

 QToolButton:pressed {
    background-color: qlineargradient(x1: 0, y1: 0, x2: 0, y2: 1,
                                      stop: 0 #red, stop: 1 #f6f7fa);
 }
/*点击时背景颜色*/

QToolButton:hover{
    background-color: qlineargradient(x1: 0, y1: 0, x2: 0, y2: 1,
```

```
                                        stop: 0 blue, stop: 1 #f6f7fa);
 }
/*鼠标划过时的背景颜色*/

 /* the subcontrols below are used only in the MenuButtonPopup mode */
/*下面的只适合 MenuButtonPopup 模式*/
/*以下对本软件无效可以猜测本软件用的不是 MenuButtonPopup 模式*/
 QToolButton::menu-button {
      border: 2px solid gray;
      border-top-right-radius: 6px;
      border-bottom-right-radius: 6px;
      /* 16px width + 4px for border = 20px allocated above */
      width: 16px;
 }

 QToolButton::menu-arrow {
      image: url(styles/start.png));
 }
/*箭头图片*/

 QToolButton::menu-arrow:open {
      top: 1px; left: 1px; /* shift it a bit */
 }
```

## 32. 定制 QToolTip

工具栏提示工具类似于 QLable，可以设置它的不同明度，默认的是跟平台有关的一个部件，可以如下定义：

```
QToolTip {
      border: 2px solid darkkhaki;     /*外框颜色*/
      background-color: #fdb933;/*背景颜色*/
      color: red;   /*字体的颜色*/
      padding: 1px;
      border-radius: 3px;
      opacity: 200;   /*透明度*/
 }
/*Warning： 提示 ToolTip 一定要在 QToolButton 样式表之前进行定义。*/
```

## 33. 定制 QTreeView

定制背景颜色交替

```
QTreeView {
      alternate-background-color: yellow;
```

docin 豆丁 www.docin.com

```
}
```

我们可以定制特别的颜色，比如鼠标划过时 item 的颜色，用 item 辅助控制器选择:

```
QTreeView {
    show-decoration-selected: 1;
}

QTreeView::item {
    border: 1px solid #d9d9d9;
    border-top-color: transparent;
    border-bottom-color: red;
}
/*一个 item 不同内容之间的分割线，每一个内容有四个方向的 border，此时表示左右为 #d9d9d9 ，下面的颜色为 red ，top 方向无颜色*/

QTreeView::item:hover {
    background: qlineargradient(x1: 0, y1: 0, x2: 0, y2: 1, stop: 0 #e7effd, stop: 1 #cbdaf1);
    border: 1px solid blue
}
/*当鼠划过时 item 的背景颜色，边缘显示 blue*/

QTreeView::item:selected {
    border: 1px solid #567dbc;
}
/*选择时外框颜色的颜色*/

QTreeView::item:selected:active{
    background: qlineargradient(x1: 0, y1: 0, x2: 0, y2: 1, stop: 0 #6ea1f1, stop: 1 #567dbc);
}
/*选中当前活动的 item 的背景色*/

QTreeView::item:selected:!active {
    background: qlineargradient(x1: 0, y1: 0, x2: 0, y2: 1, stop: 0 #6b9be8, stop: 1 #577fbf);
}
/*当选中了一个 item 时候，之后鼠标离开时点击其他非此 QTreeView 范围内时背景变的颜色*/
```

用::branch 辅助控制器来控制单独一个分支，如下：

```
QTreeView::branch {
        background: palette(base);
}

QTreeView::branch:has-siblings:!adjoins-item {
        background: #130c0e;
```

www.docin.com

```
 }
/*是指 tree 打开后没有和后面链接的地方*/

 QTreeView::branch:has-siblings:adjoins-item {
        background: red;
 }

 QTreeView::branch:!has-children:!has-siblings:adjoins-item {
        background: blue;
 }
/*没有子目录，没有兄弟目录，有链接*/

 QTreeView::branch:closed:has-children:has-siblings {
        background: pink;
 }
/*有子目录。有兄弟目录的关闭时显示 pink 色*/

 QTreeView::branch:has-children:!has-siblings:closed {
        background: gray;
 }
/*有子目录，没有兄弟目录时是 gray*/

 QTreeView::branch:open:has-children:has-siblings {
        background: magenta;
 }
/*有子目录。有兄弟目录的打开时显示 pink 颜色*/

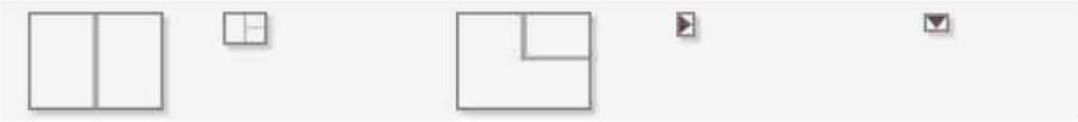
 QTreeView::branch:open:has-children:!has-siblings {
        background: green;
 }
/*有子目录没有兄弟目录的是 green*/
```

模型

| 名称 | 值 | 类型 | 描述 |
|---|---|---|---|
| R1 | | | |
| R2 | | | |
| | 0 | Double | X轴 |
| | <edit.… | 位置点… | 位置点变量 |
| | <edit.… | 索引点… | 索引点变量 |
| | <edit.… | Double[] | 三个元… |
| | 0 | Double | Sin函数 |
| c… | 0 | Refere… | |
| l… | model.… | Refere… | |
| f… | 0.1 | Refere… | |
| to | 0.9 | Refere… | |
| i… | 0.01 | Refere… | |

docin 豆丁
www.docin.com

以上讲的是各种颜色，但是更实用的是使用各种图片为我么打造更好看的 QTreeView；



Vline.png　　branch-more.png　　branch-end.png　　branch-end.png　　branch-open.png

```
QTreeView::branch:has-siblings:!adjoins-item {
        border-image: url(vline.png) 0;
}

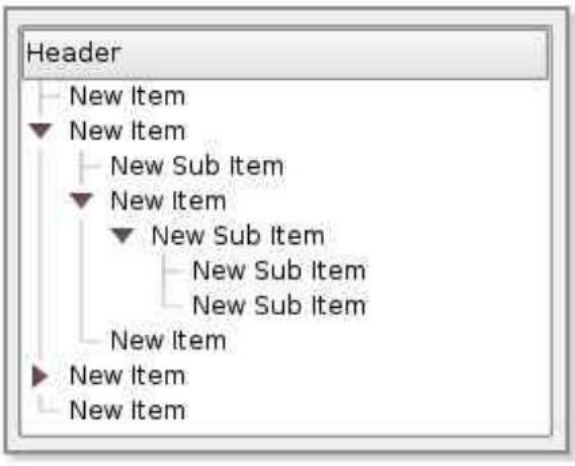
QTreeView::branch:has-siblings:adjoins-item {
        border-image: url(branch-more.png) 0;
}

QTreeView::branch:!has-children:!has-siblings:adjoins-item {
        border-image: url(branch-end.png) 0;
}

QTreeView::branch:has-children:!has-siblings:closed,
QTreeView::branch:closed:has-children:has-siblings {
            border-image: none;
            image: url(branch-closed.png);
}

QTreeView::branch:open:has-children:!has-siblings,
QTreeView::branch:open:has-children:has-siblings   {
            border-image: none;
            image: url(branch-open.png);
}
```

最后的图形效果是：




www.docin.com

## 34. QPushButton 和 images

这里我们讲一些关于 QPushButton 的样式设置的问题，当我们对他进行操作的时候，对其设置属性的时候我们往往用 **background-image** 属性设置，但是这个属性是一个很有缺陷的，例如，**background** 经常会提供隐藏在后面的 **decoration**，
那么什么是 **decoration**?
如下：

```
QTreeView {
      show-decoration-selected: 1;
 }
```

我们在例子中经常会出现这个样式属性设置。
所以 **background** 这个属性是不成熟的，考虑不周全的，如果按钮被重新设置大小，那么背景是被拉伸或者平铺，这样看起来就不那么美观了。
最好用 **border-image** 这个属性，他可以很好地解决这个问题。
如下参考：

```
QPushButton {
        color: grey;
        border-image: url(I:/aboutstylesheet/imagesForExample/example_for 001.png) 3 10 3 10;
        border-top: 3px transparent;
        border-bottom: 3px transparent;
        border-right: 10px transparent;
        border-left: 10px transparent;
      }
```

这是一个九宫格分割法，它避免了 **background-image** 这个属性的拉伸图片的处理方式，**border-image** 把图片分成九个格子，顾为九宫格分割法。

www.docin.com

## 七、参考网站

qthelp://com.trolltech.qt.482/qdoc/stylesheet.html#overview
http://doc.qt.nokia.com/4.7/stylesheet.html
http://doc.qt.nokia.com/4.7/stylesheet-syntax.html